Bölüm 4: Bağlı Listeler

C ile Veri Yapıları ve Algoritmalar

Düğüm, malloc, free

Başa · sona · araya ekleme

Silme ve arama

Ters çevirme · birleştirme

Floyd döngü tespiti

Dairesel ve çift yönlü bağlı

Başlık düğümü

Kullanılabilir düğüm listesi

Sercan KÜLCÜ · Giresun Üniversitesi · Bilgisayar Mühendisliği

GİRİŞ

Öğrenme Çıktıları

Düğüm kurmak

Kendine başvuran `struct` tanımlamak, `malloc` sonucunu denetlemek, `free` ile iade etmek.

Bağ disiplini

Başa, sona, araya ve sıralı eklemeyi bağı kaybetmeden, doğru sırayla yapmak.

Güvenli silme

Bellek sızıntısı ve sarkan işaretçi (dangling pointer) oluşturmadan düğüm silmek.

Klasik yordamlar

Ters çevirme, birleştirme, orta düğüm ve Floyd döngü tespitini O(1) ek bellekle yazmak.

Liste türleri

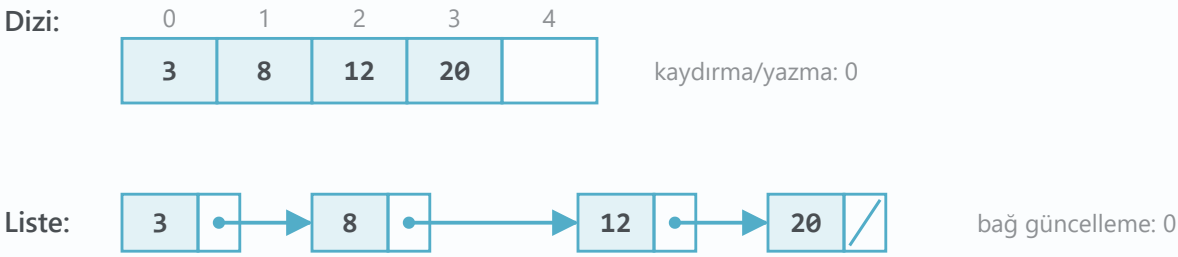
Dairesel, çift yönlü bağlı ve başlık düğümlü listeleri amaca göre seçmek.

Bellek yönetimi

Kullanılabilir düğüm listesiyle (available space list) düğümleri yeniden kullanmak.

NEDEN LİSTE?

Dizi yetmediğinde: kaydırma yerine bağ



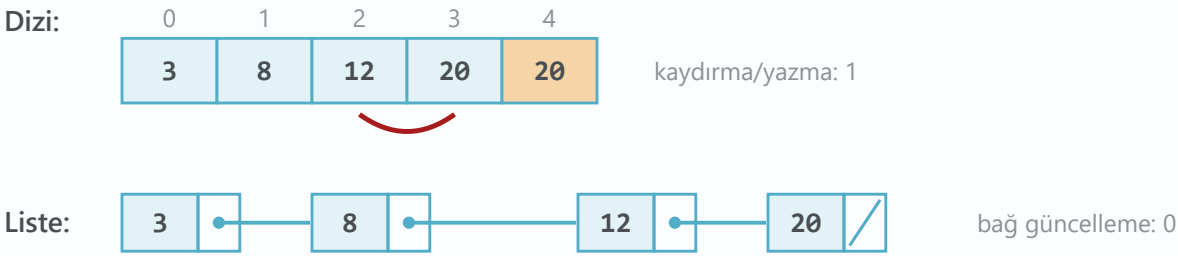
8'den sonra 10 eklenecek. Dizide yer açmak için arkadaki öğeler kaydırılır; listede yalnızca bağlar değişir.

Adım 1 / 6

Hocadan "Pointerlar bu yaşlarda öğrenilmezse 4. sınıfta bile öğrenilmemiş olur."

NEDEN LİSTE?

Dizi yetmediğinde: kaydırma yerine bağ



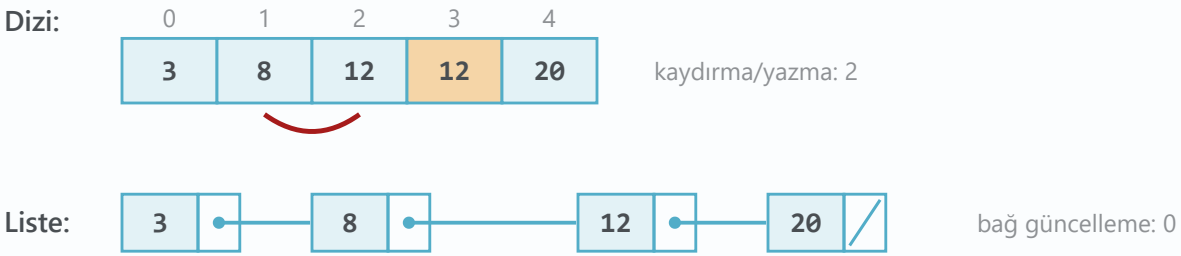
Dizi: 20 bir sağa kaydı (1. taşıma).

Adım 2 / 6

Hocadan “Pointerlar bu yaşlarda öğrenilmezse 4. sınıfta bile öğrenilmemiş olur.”

NEDEN LİSTE?

Dizi yetmediğinde: kaydırma yerine bağ

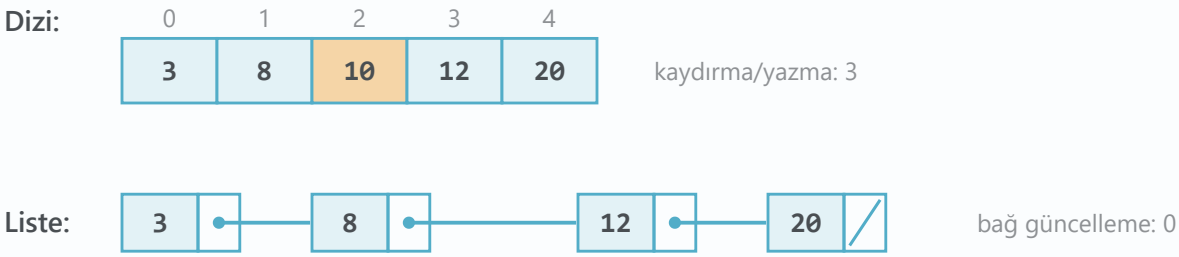


Dizi: 12 bir sağa kaydı (2. taşıma). n öğeli dizide en kötü durumda n taşıma: **O(n)**.

Hocadan "Pointerlar bu yaşlarda öğrenilmezse 4. sınıfta bile öğrenilmemiş olur."

NEDEN LİSTE?

Dizi yetmediğinde: kaydırma yerine bağ



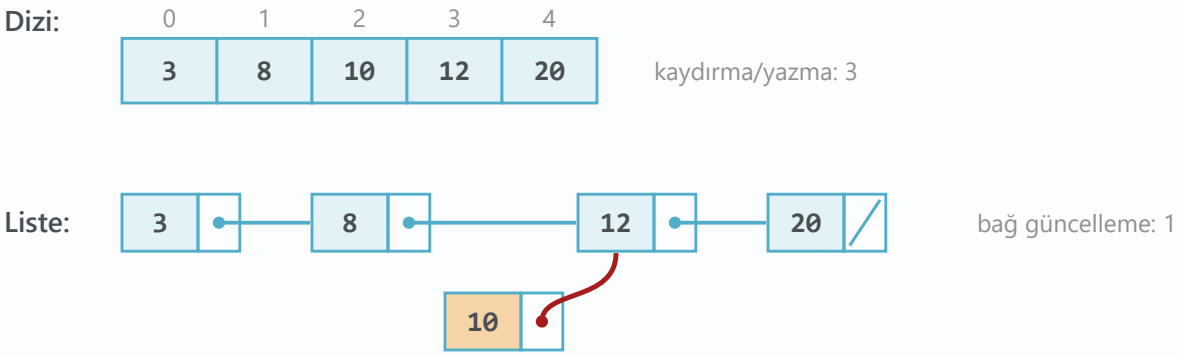
Dizi: boşalan hücreye 10 yazıldı.

Adım 4 / 6

Hocadan “Pointerlar bu yaşlarda öğrenilmezse 4. sınıfta bile öğrenilmemiş olur.”

NEDEN LİSTE?

Dizi yetmediğinde: kaydırma yerine bağ



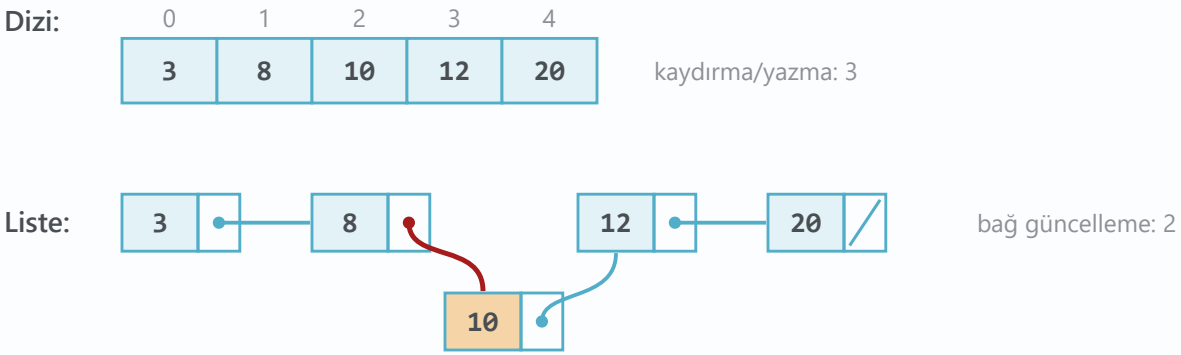
Liste: yeni düğüm ayrıldı, `yeni→bag` 12'yi gösterdi. Hiçbir öğe yer değiştirmedi.

Adım 5 / 6

Hocadan "Pointerlar bu yaşlarda öğrenilmezse 4. sınıfta bile öğrenilmemiş olur."

NEDEN LİSTE?

Dizi yetmediğinde: kaydırma yerine bağ



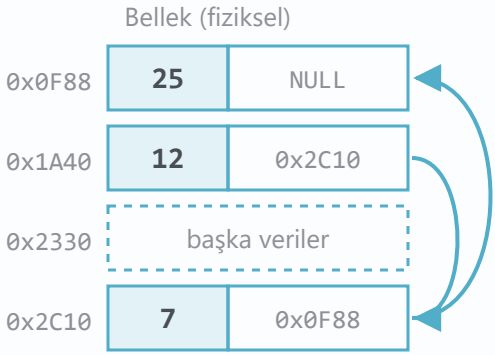
Liste: `8→bag = yeni`. Konum biliniyorsa yalnızca iki bağ: **O(1)**.

Adım 6 / 6

Hocadan "Pointerlar bu yaşlarda öğrenilmezse 4. sınıfta bile öğrenilmemiş olur."

NEDEN LİSTE?

Düğümler bellekte yan yana olmak zorunda değil




```
bas = 0x1A40
```



Sıra, adreslerden değil bağlardan gelir.

Düğüm = veri + bağ

`bag` (link) alanı sonraki düğümün **adresini** tutar; son düğümde `NULL`.

Rastgele erişim yok

i. düğümün adresi hesaplanamaz; baştan bağlar izlenir: O(i).

Adres nedir?

Eski x86'da adres iki parçalıydı (bölüt + bölüt içi adres): bölütlü bellek. Günümüzde düz adres uzayı.

NEDEN LİSTE?

İşaretçi (pointer) hatırlatması

```
int x = 5;
int *p = &x;        // p, x'in adresini tutar
*p = 7;             // p'nin gösterdiğine yaz: x == 7

oge_gost q = NULL; // henüz hiçbir düğüm yok
q = malloc(sizeof *q);
if (q != NULL) {
    q->deger = 3;  // (*q).deger = 3
    q->bag = NULL;
}
free(q);            // iade; q artık sarkan işaretçi
q = NULL;
```

`&` ve `*`

`&x`: x'in adresi · `*p`: p'nin gösterdiği nesne.

`→`

`q→deger`, `(*q).deger` ile aynıdır; öncelik için parantez şart.

NULL

Hiçbir yeri göstermeyen işaretçi; boş listenin gösterimi.

Hocadan "İki derstir konu bırakılıp C ve pointerlar anlatıldı. Pointerları çok iyi bildiğini düşünüp anlatılanı yine de anlamamak olabilir; olağan bir durum."

NEDEN LİSTE?

Düğüm ayırmak: `malloc`, denetim, `free`

```
oge_gost yeni_oge(int x) {
    oge_gost p = malloc(sizeof *p);
    if (p == NULL) {
        fprintf(stderr, "Bellek yetmiyor yetmiyoor...\n");
        exit(1);
    }
    p->deger = x;
    p->bag = NULL;
    return p;
}
```

`sizeof *p` ve NULL

Türü tekrar yazmadan doğru boyut. Bellek bitince `malloc` NULL döner; örnek ileti: *Bellek yetmiyor yetmiyoor...*

free kuralı

Her `malloc` için tam bir `free`; iki kez free ya da free sonrası erişim tanımsız davranıştır.

Hocadan "Her pointer'a malloc'la yer ayırmak gerekmiyor, yok öyle bir şey!"

Gezinen işaretçiler (`p = p→bag`) var olan düğümleri gösterir; onlar için malloc yapılmaz.

Düğüm: kendine başvuran yapı (self-referential struct)

```c
typedef struct oge *oge_gost;  // önce işaretçi türü

struct oge {
    int deger;       // veri alanı
    oge_gost bag;    // sonraki düğümün adresi
};

oge_gost liste = NULL;  // boş liste
```

Neden önce typedef?

Yapı kendi türünden bir işaretçiyi içerir; eksik (incomplete) türe işaretçi tanımlamak serbesttir.

Adlandırma

İşaretçi türüne `list_gosterge` ya da `oge_gost` gibi bir ad verilebilir; burada `oge_gost`.

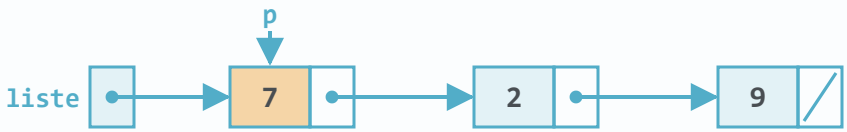


struct oge: veri alanı + bag alanı

Sık yapılan hata: `typedef struct liste { … };` yazıp typedef'e ad vermemek. Bu durumda `typedef` atılmalı ya da sona tür adı yazılmalıdır.

TEK YÖNLÜ BAĞLI LİSTE · ADIM ADIM

Listede gezinme: `p = p→bag`



`yazdir(liste)` : işaretçi **değerle** geldi; p'yi ilerletmek main'deki `liste` 'yi değiştirmez.

Adım 1 / 8

```
▶ void yazdir(oge_gost p) {
2     for (; p != NULL; p = p->bag)
3         printf("%d ", p->deger);
4     printf("\n");
5 }
```

DEĞİŞKENLER

`p = &düğüm(7)`

ÇIKTI

Uzunluk aynı kalıpla sayılır: `for (; p ≠ NULL; p = p→bag) n++;`

Listede gezinme: `p = p→bag`



p ilk düğümde: 7 yazılır.


```
1 void yazdir(oge_gost p) {
2     for (; p != NULL; p = p->bag)
▶         printf("%d ", p->deger);
4     printf("\n");
5 }
```

DEĞİŞKENLER

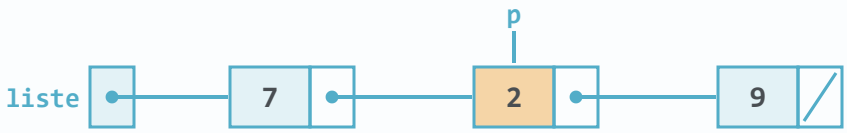
`p = &düğüm(7)`

ÇIKTI

```
7
```

Uzunluk aynı kalıpla sayılır: `for (; p ≠ NULL; p = p→bag) n++;`

Listede gezinme: `p = p→bag`



`p = p→bag` : ikinci düğüm.


```
1 void yazdir(oge_gost p) {
▶       for (; p != NULL; p = p->bag)
3           printf("%d ", p->deger);
4       printf("\n");
5 }
```

DEĞİŞKENLER

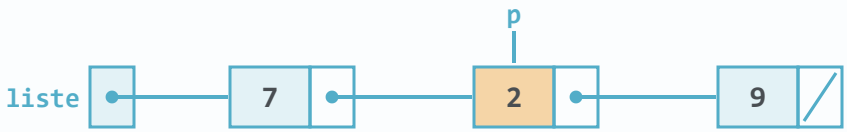
`p = &düğüm(2)`

ÇIKTI

```
7
```

Uzunluk aynı kalıpla sayılır: `for (; p ≠ NULL; p = p→bag) n++;`

Listede gezinme: `p = p→bag`



2 yazılır.

Adım 4 / 8

```
1 void yazdir(oge_gost p) {
2     for (; p != NULL; p = p->bag)
          printf("%d ", p->deger);
4     printf("\n");
5 }
```

DEĞİŞKENLER

`p = &düğüm(2)`

ÇIKTI

```
7 2
```

Uzunluk aynı kalıpla sayılır: `for (; p ≠ NULL; p = p→bag) n++;`

Listede gezinme: `p = p→bag`



Üçüncü düğüm.

Adım 5 / 8

```
1 void yazdir(oge_gost p) {
▶       for (; p != NULL; p = p->bag)
3           printf("%d ", p->deger);
4       printf("\n");
5 }
```

DEĞİŞKENLER

`p = &düğüm(9)`

ÇIKTI

```
7 2
```

Uzunluk aynı kalıpla sayılır: `for (; p ≠ NULL; p = p→bag) n++;`

Listede gezinme: `p = p→bag`



9 yazılır.


```
1 void yazdir(oge_gost p) {
2     for (; p != NULL; p = p->bag)
          printf("%d ", p->deger);
4     printf("\n");
5 }
```

DEĞİŞKENLER

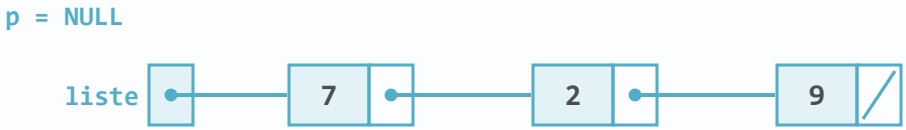
`p = &düğüm(9)`

ÇIKTI

```
7 2 9
```

Uzunluk aynı kalıpla sayılır: `for (; p ≠ NULL; p = p→bag) n++;`

Listede gezinme: `p = p→bag`



`p == NULL` : döngü biter.

Adım 7 / 8

```
1 void yazdir(oge_gost p) {
▶       for (; p != NULL; p = p->bag)
3           printf("%d ", p->deger);
4       printf("\n");
5   }
```

DEĞİŞKENLER

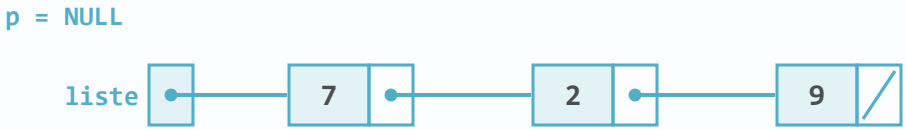
`p = NULL`

ÇIKTI

```
7 2 9
```

Uzunluk aynı kalıpla sayılır: `for (; p ≠ NULL; p = p→bag) n++;`

Listede gezinme: `p = p→bag`



Satır sonu. n düğüm için n tur: **O(n)**.

Adım 8 / 8

```
1 void yazdir(oge_gost p) {
2     for (; p != NULL; p = p->bag)
3         printf("%d ", p->deger);
▶     printf("\n");
5 }
```

DEĞİŞKENLER

`p = NULL`

ÇIKTI

```
7 2 9
```

Uzunluk aynı kalıpla sayılır: `for (; p ≠ NULL; p = p→bag) n++;`

Başa ekleme ve adresle aktarım

```
bool basa_ekle(oge_gost *bas, int x) {
    oge_gost yeni = malloc(sizeof *yeni);
    if (yeni == NULL) return false;
    yeni->deger = x;
    yeni->bag = *bas;
    *bas = yeni;
    return true;
}

oge_gost liste = NULL;
basa_ekle(&liste, 2);   // 2
basa_ekle(&liste, 7);   // 7 -> 2
```

Neden `oge_gost *bas`?

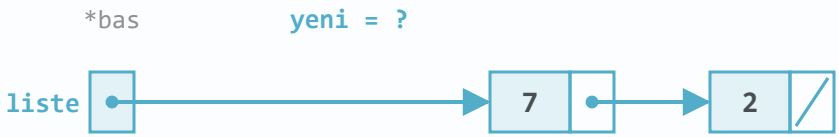
Baş değişecek. Değerle gelseydi yalnızca yerel kopya değişirdi, main'deki `liste` eski kalırdı.

Sıra önemli

Önce `*bas = yeni` yazılsaydı düğüm kendine bağlanırdı, listenin kalanı kaybolurdu. Maliyet **O(1)**.

Hocadan "Normal şartlarda yordamın formal parametre listesinde '*parametre' olarak gözüken değişken, yordam içinde de '*parametre' olarak kullanılmalıdır."

Kod İzleme: `basa_ekle(&liste, 5)`



Çağrı: `basa_ekle(&liste, 5)` . `bas` , main'deki `liste` değişkeninin **adresidir**; liste 7 → 2.

Adım 1 / 7

```
▶ bool basa_ekle(oge_gost *bas, int x) {
2     oge_gost yeni = malloc(sizeof *yeni);
3     if (yeni == NULL) return false;  // bellek yok
4     yeni->deger = x;
5     yeni->bag = *bas;     // önce eski başı koru
6     *bas = yeni;          // sonra başı değiştir
7     return true;
8 }
```

DEĞİŞKENLER

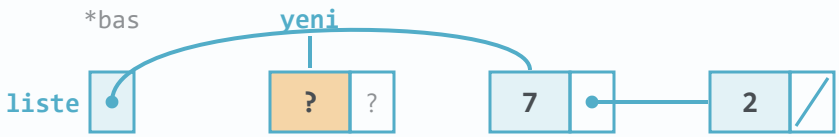
`bas = &liste` `x = 5` `yeni = ?` `*bas = &düğüm(7)`

ÇAĞRI YIĞINI

`basa_ekle(&liste, 5)`

`main()`

Kod İzleme: `basa_ekle(&liste, 5)`



`malloc` heap bellekten bir düğüm ayırdı. Alanları henüz **çöp** değer taşıyor.

Adım 2 / 7

```
1 bool basa_ekle(oge_gost *bas, int x) {
▶     oge_gost yeni = malloc(sizeof *yeni);
3     if (yeni == NULL) return false;  // bellek yok
4     yeni->deger = x;
5     yeni->bag = *bas;     // önce eski başı koru
6     *bas = yeni;          // sonra başı değiştir
7     return true;
8 }
```

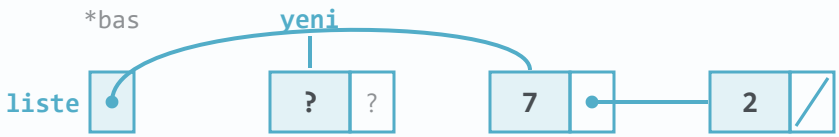
DEĞİŞKENLER

- `bas = &liste`
- `x = 5`
- `yeni = &düğüm(?)`
- `*bas = &düğüm(7)`

ÇAĞRI YIĞINI

- `basa_ekle(&liste, 5)`
- `main()`

Kod İzleme: `basa_ekle(&liste, 5)`



`yeni ≠ NULL` : bellek ayrılabildi, devam. (Ayrılamasaydı liste hiç bozulmadan `false` dönecekti.)

Adım 3 / 7

```
1 bool basa_ekle(oge_gost *bas, int x) {
2     oge_gost yeni = malloc(sizeof *yeni);
▶     if (yeni == NULL) return false;  // bellek yok
4     yeni->deger = x;
5     yeni->bag = *bas;     // önce eski başı koru
6     *bas = yeni;          // sonra başı değiştir
7     return true;
8 }
```

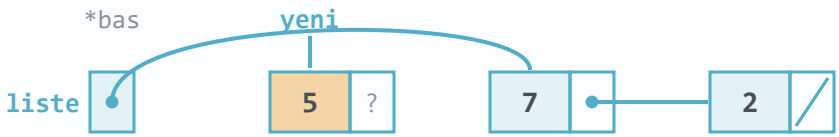
DEĞİŞKENLER

- `bas = &liste`
- `x = 5`
- `yeni = &düğüm(?)`
- `*bas = &düğüm(7)`

ÇAĞRI YIĞINI

- `basa_ekle(&liste, 5)`
- `main()`

Kod İzleme: `basa_ekle(&liste, 5)`



Veri alanı dolduruldu: `yeni→deger = 5`.

Adım 4 / 7

```
1 bool basa_ekle(oge_gost *bas, int x) {
2     oge_gost yeni = malloc(sizeof *yeni);
3     if (yeni == NULL) return false;  // bellek yok
▶     yeni->deger = x;
5     yeni->bag = *bas;     // önce eski başı koru
6     *bas = yeni;          // sonra başı değiştir
7     return true;
8 }
```

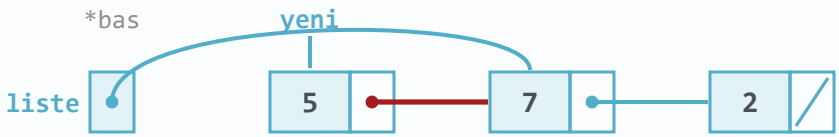
DEĞİŞKENLER

- `bas = &liste`
- `x = 5`
- `yeni = &düğüm(5)`
- `*bas = &düğüm(7)`

ÇAĞRI YIĞINI

- `basa_ekle(&liste, 5)`
- `main()`

Kod İzleme: `basa_ekle(&liste, 5)`



Önce eski baş korunur: `yeni→bag = *bas` → yeni düğüm 7'yi gösterir. `liste` hâlâ 7'de.

Adım 5 / 7

```
1 bool basa_ekle(oge_gost *bas, int x) {
2     oge_gost yeni = malloc(sizeof *yeni);
3     if (yeni == NULL) return false;  // bellek yok
4     yeni->deger = x;
▶     yeni->bag = *bas;     // önce eski başı koru
6     *bas = yeni;          // sonra başı değiştir
7     return true;
8 }
```

DEĞİŞKENLER

`bas = &liste` `x = 5`

`yeni = &düğüm(5)`

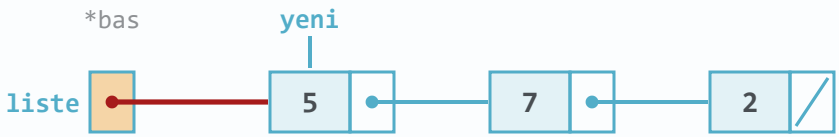
`*bas = &düğüm(7)`

ÇAĞRI YIĞINI

`basa_ekle(&liste, 5)`

`main()`

Kod İzleme: `basa_ekle(&liste, 5)`



Sonra baş değişir: `*bas = yeni`. Adresle aktarım sayesinde main'deki `liste` güncellendi.

Adım 6 / 7

```
1 bool basa_ekle(oge_gost *bas, int x) {
2     oge_gost yeni = malloc(sizeof *yeni);
3     if (yeni == NULL) return false;  // bellek yok
4     yeni->deger = x;
5     yeni->bag = *bas;     // önce eski başı koru
▶     *bas = yeni;          // sonra başı değiştir
7     return true;
8 }
```

DEĞİŞKENLER

`bas = &liste` `x = 5`

`yeni = &düğüm(5)`

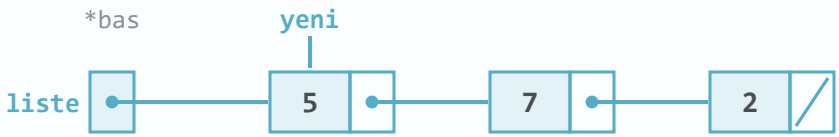
`*bas = &düğüm(5)`

ÇAĞRI YIĞINI

`basa_ekle(&liste, 5)`

`main()`

Kod İzleme: `basa_ekle(&liste, 5)`



Liste artık 5 → 7 → 2. Liste uzunluğundan bağımsız, sabit sayıda iş: **O(1)**.

Adım 7 / 7

```
1 bool basa_ekle(oge_gost *bas, int x) {
2     oge_gost yeni = malloc(sizeof *yeni);
3     if (yeni == NULL) return false;  // bellek yok
4     yeni->deger = x;
5     yeni->bag = *bas;     // önce eski başı koru
6     *bas = yeni;          // sonra başı değiştir
▶     return true;
8 }
```

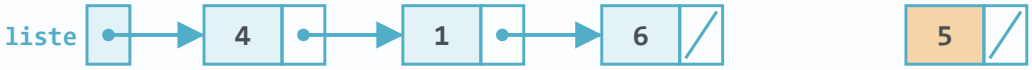
DEĞİŞKENLER

- `bas = &liste`
- `x = 5`
- `yeni = &düğüm(5)`
- `*bas = &düğüm(5)`

ÇAĞRI YIĞINI

- `basa_ekle(&liste, 5)`
- `main()`

Sona ekleme: son düğümü bulmak O(n)



Yeni düğüm (5) hazır; `bag = NULL` çünkü son olacak.

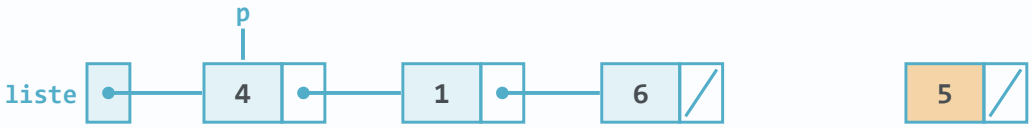
Adım 1 / 4

```
1  bool sona_ekle(oge_gost *bas, int x) {
2      oge_gost p, yeni = malloc(sizeof *yeni);
3      if (yeni == NULL) return false;
▶      yeni->deger = x;
▶      yeni->bag = NULL;              // yeni son düğüm
6      if (*bas == NULL) { *bas = yeni; return true; }
7      for (p = *bas; p->bag != NULL; p = p->bag)
8          ;                          // son düğümü bul
9      p->bag = yeni;
10     return true;
11 }
```

DEĞİŞKENLER

`yeni = &düğüm(5)` `p = ?`

Sona ekleme: son düğümü bulmak O(n)



Liste boş değil: `p` baştan başlar.

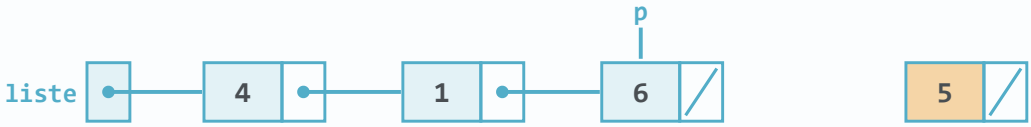
Adım 2 / 4

```
1  bool sona_ekle(oge_gost *bas, int x) {
2      oge_gost p, yeni = malloc(sizeof *yeni);
3      if (yeni == NULL) return false;
4      yeni->deger = x;
5      yeni->bag = NULL;              // yeni son düğüm
6      if (*bas == NULL) { *bas = yeni; return true; }
▶      for (p = *bas; p->bag != NULL; p = p->bag)
8          ;                          // son düğümü bul
9      p->bag = yeni;
10     return true;
11 }
```

DEĞİŞKENLER

`yeni = &düğüm(5)` `p = &düğüm(4)`

Sona ekleme: son düğümü bulmak O(n)



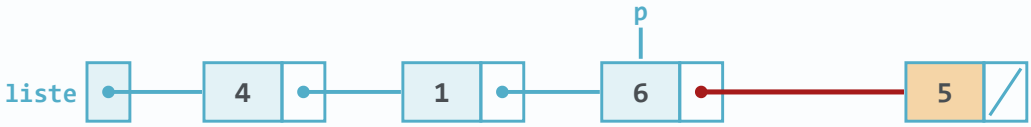
`p→bag == NULL` olana dek ilerler: son düğüm 6. Bu yürüyüş **O(n)**.


```
1  bool sona_ekle(oge_gost *bas, int x) {
2      oge_gost p, yeni = malloc(sizeof *yeni);
3      if (yeni == NULL) return false;
4      yeni->deger = x;
5      yeni->bag = NULL;              // yeni son düğüm
6      if (*bas == NULL) { *bas = yeni; return true; }
▶      for (p = *bas; p->bag != NULL; p = p->bag)
8          ;                          // son düğümü bul
9      p->bag = yeni;
10     return true;
11 }
```

DEĞİŞKENLER

`yeni = &düğüm(5)` `p = &düğüm(6)`

Sona ekleme: son düğümü bulmak O(n)



`p->bag = yeni`. Sürekli sona ekleniyorsa bir `son` işaretçisi tutmak işi O(1) yapar (bağlı kuyruk bu yolla gerçeklenir).

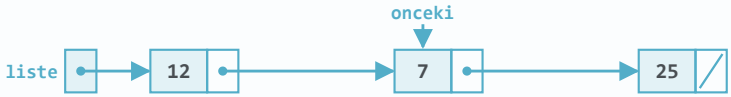
Adım 4 / 4

```
1  bool sona_ekle(oge_gost *bas, int x) {
2      oge_gost p, yeni = malloc(sizeof *yeni);
3      if (yeni == NULL) return false;
4      yeni->deger = x;
5      yeni->bag = NULL;              // yeni son düğüm
6      if (*bas == NULL) { *bas = yeni; return true; }
7      for (p = *bas; p->bag != NULL; p = p->bag)
8          ;                          // son düğümü bul
▶      p->bag = yeni;
10     return true;
11 }
```

DEĞİŞKENLER

`yeni = &düğüm(5)` `p = &düğüm(6)`

Araya ekleme: önce sonraki korunur, sonra bağlanır



12 → 7 → 25; `onceki` = düğüm(7). 7'nin peşine 9 eklenecek.

Adım 1 / 4

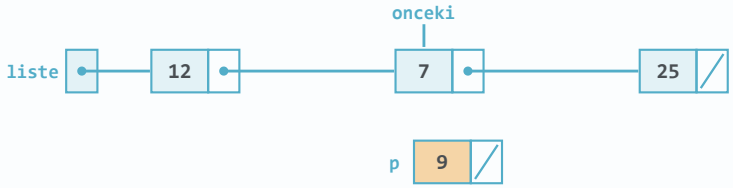
```
1 // onceki'nin peşine x değerli yeni düğüm ekler
▶ oge_gost sonrasina_ekle(oge_gost onceki, int x) {
3     oge_gost p = yeni_oge(x);  // malloc + denetim
4     if (onceki != NULL) {      // araya ya da sona
5         p->bag = onceki->bag;
6         onceki->bag = p;
7     }                          // yoksa tek başına
8     return p;
9 }
```

DEĞİŞKENLER

`onceki = &düğüm(7)` `x = 9` `p = ?`

Polinom çarpımındaki `YeniTerimEkle(Ust, Katsayi, Onceki)` yordamının genelleştirilmiş hâli.

Araya ekleme: önce sonraki korunur, sonra bağlanır



`yeni_oge(9)` : düğüm ayrıldı, `bag = NULL` .

Adım 2 / 4

```
1 // onceki'nin peşine x değerli yeni düğüm ekler
2 oge_gost sonrasina_ekle(oge_gost onceki, int x) {
▶     oge_gost p = yeni_oge(x);  // malloc + denetim
4     if (onceki != NULL) {      // araya ya da sona
5         p->bag = onceki->bag;
6         onceki->bag = p;
7     }                          // yoksa tek başına
8     return p;
9 }
```

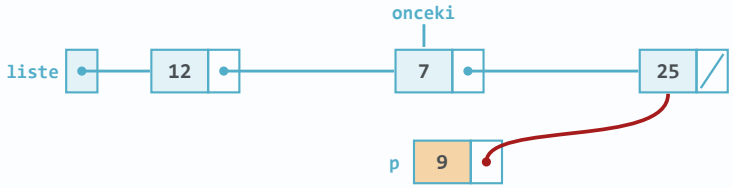
DEĞİŞKENLER

`onceki = &düğüm(7)` `x = 9`

`p = &düğüm(9)`

Polinom çarpımındaki `YeniTerimEkle(Ust, Katsayi, Onceki)` yordamının genelleştirilmiş hâli.

Araya ekleme: önce sonraki korunur, sonra bağlanır



`p→bag = onceki→bag` : 25'in adresi yeni düğüme kopyalandı; hiçbir şey kaybolmadı.

Adım 3 / 4

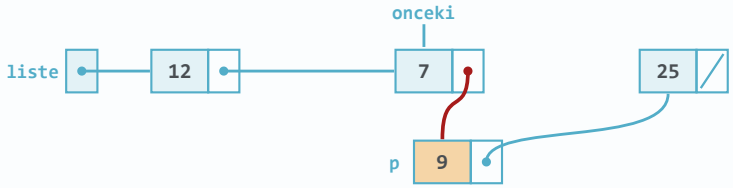
```
1 // onceki'nin peşine x değerli yeni düğüm ekler
2 oge_gost sonrasina_ekle(oge_gost onceki, int x) {
3     oge_gost p = yeni_oge(x);  // malloc + denetim
4     if (onceki != NULL) {      // araya ya da sona
▶         p->bag = onceki->bag;
6         onceki->bag = p;
7     }                          // yoksa tek başına
8     return p;
9 }
```

DEĞİŞKENLER

`onceki = &düğüm(7)` `x = 9` `p = &düğüm(9)`

Polinom çarpımındaki `YeniTerimEkle(Ust, Katsayi, Onceki)` yordamının genelleştirilmiş hâli.

Araya ekleme: önce sonraki korunur, sonra bağlanır



`onceki->bag = p` : 12 → 7 → 9 → 25. Düğüm biliniyorsa **O(1)**.

Adım 4 / 4

```
1 // onceki'nin peşine x değerli yeni düğüm ekler
2 oge_gost sonrasina_ekle(oge_gost onceki, int x) {
3     oge_gost p = yeni_oge(x);  // malloc + denetim
4     if (onceki != NULL) {      // araya ya da sona
5         p->bag = onceki->bag;
▶         onceki->bag = p;
7     }                          // yoksa tek başına
8     return p;
9 }
```

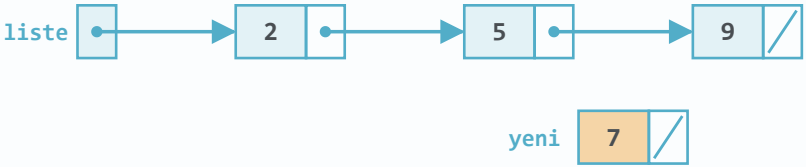
DEĞİŞKENLER

`onceki = &düğüm(7)` `x = 9`

`p = &düğüm(9)`

Polinom çarpımındaki `YeniTerimEkle(Ust, Katsayi, Onceki)` yordamının genelleştirilmiş hâli.

Kod İzleme: `sirali_ekle()` — 7 ekle



Sıralı liste 2 → 5 → 9'a 7 eklenecek. Yeni düğüm (malloc ile) hazır; `yeni→bag` henüz bağlı değil.

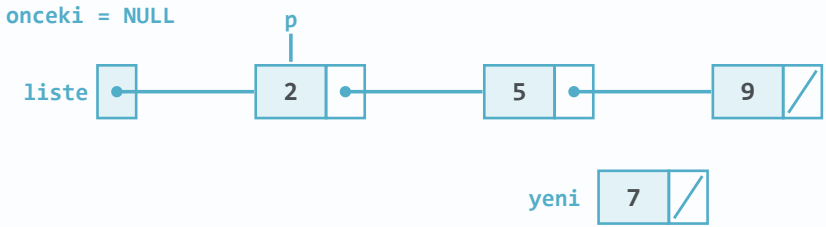
Adım 1 / 12

```
▶ void sirali_ekle(oge_gost *bas, oge_gost yeni) {
2     oge_gost onceki = NULL, p = *bas;
3     while (p != NULL && p->deger < yeni->deger) {
4         onceki = p;
5         p = p->bag;
6     }
7     yeni->bag = p;
8     if (onceki == NULL) *bas = yeni;  // başa
9     else onceki->bag = yeni;          // araya/sona
10 }
```

DEĞİŞKENLER

`yeni = &düğüm(7)` `onceki = ?` `p = ?` `*bas = &düğüm(2)`

Kod İzleme: `sirali_ekle()` — 7 ekle



`p` baştan başlar; `onceki`, p'nin bir gerisinden gelecek (şimdilik NULL).

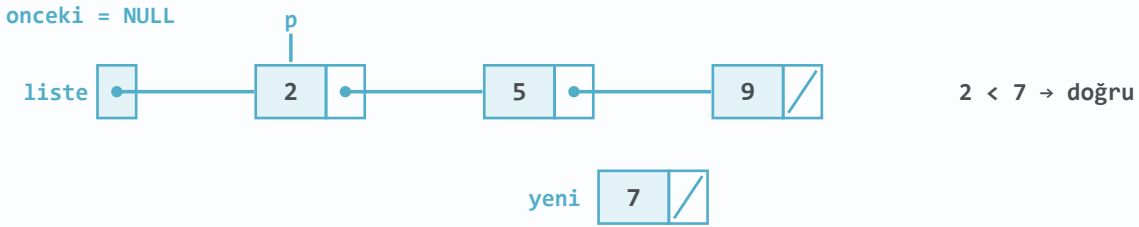
Adım 2 / 12

```
 1 void sirali_ekle(oge_gost *bas, oge_gost yeni) {
 2     oge_gost onceki = NULL, p = *bas;
 3     while (p != NULL && p->deger < yeni->deger) {
 4         onceki = p;
 5         p = p->bag;
 6     }
 7     yeni->bag = p;
 8     if (onceki == NULL) *bas = yeni;  // başa
 9     else onceki->bag = yeni;          // araya/sona
10 }
```

DEĞİŞKENLER

- `yeni = &düğüm(7)`
- `onceki = NULL`
- `p = &düğüm(2)`
- `*bas = &düğüm(2)`

Kod İzleme: `sirali_ekle()` — 7 ekle



Koşul: `2 < 7 → doğru`. Daha ileri gitmeliyiz.

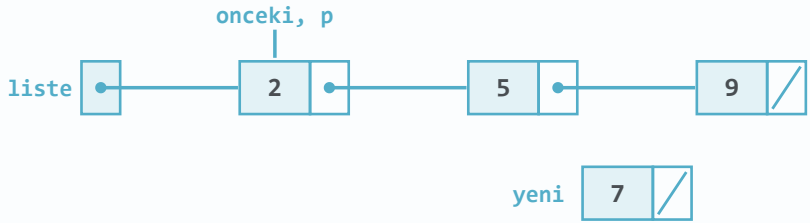
Adım 3 / 12

```
1  void sirali_ekle(oge_gost *bas, oge_gost yeni) {
2      oge_gost onceki = NULL, p = *bas;
▶      while (p != NULL && p->deger < yeni->deger) {
4          onceki = p;
5          p = p->bag;
6      }
7      yeni->bag = p;
8      if (onceki == NULL) *bas = yeni;  // başa
9      else onceki->bag = yeni;          // araya/sona
10 }
```

DEĞİŞKENLER

- `yeni = &düğüm(7)`
- `onceki = NULL`
- `p = &düğüm(2)`
- `*bas = &düğüm(2)`

Kod İzleme: `sirali_ekle()` — 7 ekle



`onceki = p` → düğüm(2).

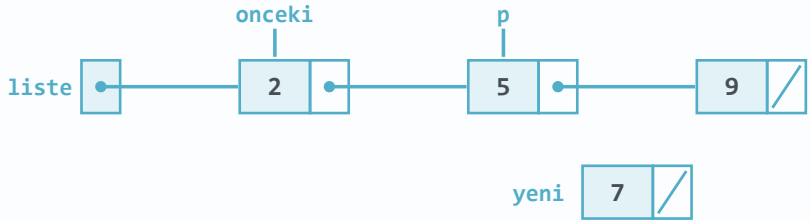
Adım 4 / 12

```
1  void sirali_ekle(oge_gost *bas, oge_gost yeni) {
2      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger < yeni->deger) {
▶          onceki = p;
5          p = p->bag;
6      }
7      yeni->bag = p;
8      if (onceki == NULL) *bas = yeni;  // başa
9      else onceki->bag = yeni;          // araya/sona
10 }
```

DEĞİŞKENLER

- `yeni = &düğüm(7)`
- `onceki = &düğüm(2)`
- `p = &düğüm(2)`
- `*bas = &düğüm(2)`

Kod İzleme: `sirali_ekle()` — 7 ekle



`p = p→bag` → &düğüm(5).

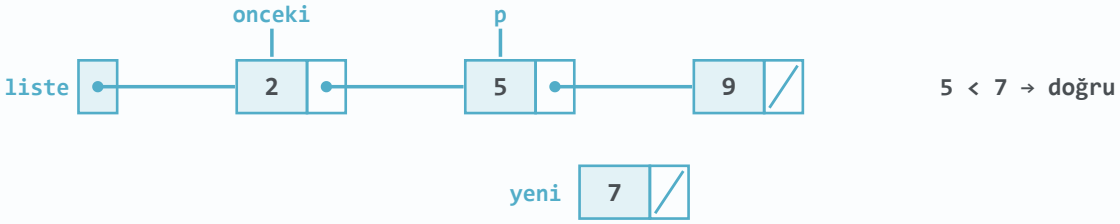
Adım 5 / 12

```
1  void sirali_ekle(oge_gost *bas, oge_gost yeni) {
2      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger < yeni->deger) {
4          onceki = p;
▶          p = p->bag;
6      }
7      yeni->bag = p;
8      if (onceki == NULL) *bas = yeni;  // başa
9      else onceki->bag = yeni;          // araya/sona
10 }
```

DEĞİŞKENLER

- `yeni` = **&düğüm(7)**
- `onceki` = **&düğüm(2)**
- `p` = **&düğüm(5)**
- `*bas` = **&düğüm(2)**

Kod İzleme: `sirali_ekle()` — 7 ekle



Koşul: `5 < 7 → doğru`. Daha ileri gitmeliyiz.

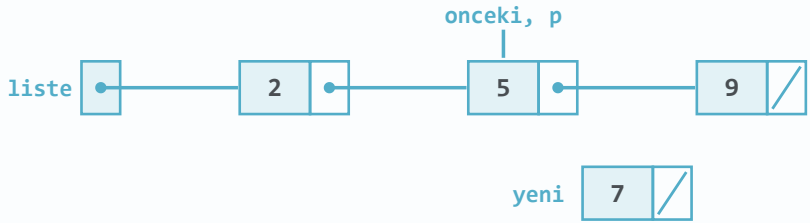
Adım 6 / 12

```
1  void sirali_ekle(oge_gost *bas, oge_gost yeni) {
2      oge_gost onceki = NULL, p = *bas;
▶      while (p != NULL && p->deger < yeni->deger) {
4          onceki = p;
5          p = p->bag;
6      }
7      yeni->bag = p;
8      if (onceki == NULL) *bas = yeni;  // başa
9      else onceki->bag = yeni;          // araya/sona
10 }
```

DEĞİŞKENLER

- `yeni = &düğüm(7)`
- `onceki = &düğüm(2)`
- `p = &düğüm(5)`
- `*bas = &düğüm(2)`

Kod İzleme: `sirali_ekle()` — 7 ekle



`onceki = p` → düğüm(5).

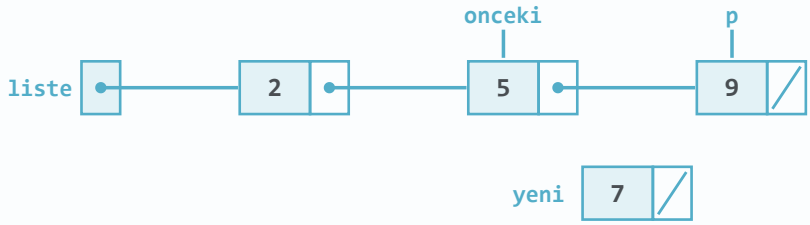
Adım 7 / 12

```
1  void sirali_ekle(oge_gost *bas, oge_gost yeni) {
2      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger < yeni->deger) {
▶          onceki = p;
5          p = p->bag;
6      }
7      yeni->bag = p;
8      if (onceki == NULL) *bas = yeni;  // başa
9      else onceki->bag = yeni;          // araya/sona
10 }
```

DEĞİŞKENLER

- `yeni = &düğüm(7)`
- `onceki = &düğüm(5)`
- `p = &düğüm(5)`
- `*bas = &düğüm(2)`

Kod İzleme: `sirali_ekle()` — 7 ekle



`p = p→bag` → &düğüm(9).

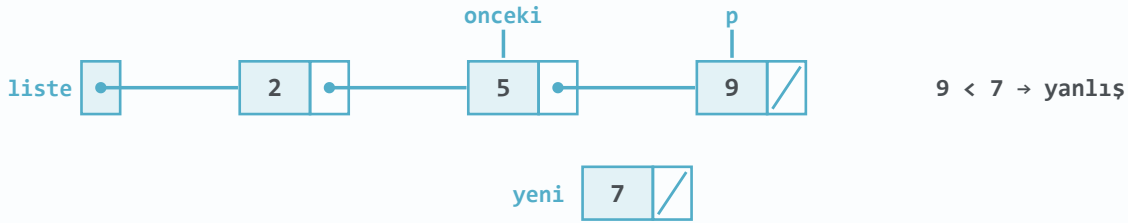
Adım 8 / 12

```
1  void sirali_ekle(oge_gost *bas, oge_gost yeni) {
2      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger < yeni->deger) {
4          onceki = p;
▶          p = p->bag;
6      }
7      yeni->bag = p;
8      if (onceki == NULL) *bas = yeni;  // başa
9      else onceki->bag = yeni;          // araya/sona
10 }
```

DEĞİŞKENLER

- `yeni` = **&düğüm(7)**
- `onceki` = **&düğüm(5)**
- `p` = **&düğüm(9)**
- `*bas` = **&düğüm(2)**

Kod İzleme: `sirali_ekle()` — 7 ekle



Koşul: `9 < 7 → yanlış`. Ekleme yeri bulundu: **onceki ile p arası**.

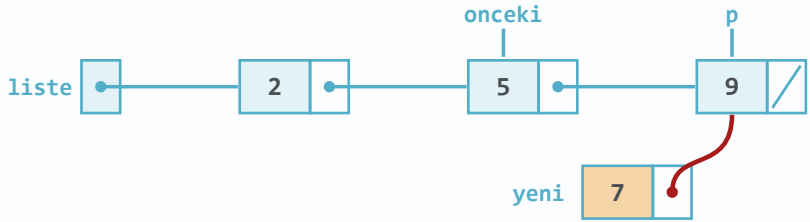
Adım 9 / 12

```
1  void sirali_ekle(oge_gost *bas, oge_gost yeni) {
2      oge_gost onceki = NULL, p = *bas;
▶      while (p != NULL && p->deger < yeni->deger) {
4          onceki = p;
5          p = p->bag;
6      }
7      yeni->bag = p;
8      if (onceki == NULL) *bas = yeni;  // başa
9      else onceki->bag = yeni;          // araya/sona
10 }
```

DEĞİŞKENLER

- `yeni = &düğüm(7)`
- `onceki = &düğüm(5)`
- `p = &düğüm(9)`
- `*bas = &düğüm(2)`

Kod İzleme: `sirali_ekle()` — 7 ekle



Önce yeni düğüm p'ye bağlanır: `yeni→bag = p` (9'un adresi kaybolmasın).

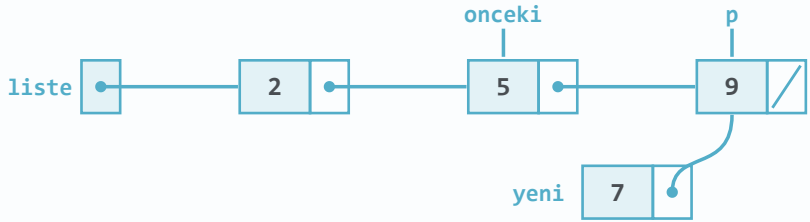
Adım 10 / 12

```
1  void sirali_ekle(oge_gost *bas, oge_gost yeni) {
2      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger < yeni->deger) {
4          onceki = p;
5          p = p->bag;
6      }
▶      yeni->bag = p;
8      if (onceki == NULL) *bas = yeni;  // başa
9      else onceki->bag = yeni;          // araya/sona
10 }
```

DEĞİŞKENLER

- `yeni = &düğüm(7)`
- `onceki = &düğüm(5)`
- `p = &düğüm(9)`
- `*bas = &düğüm(2)`

Kod İzleme: `sirali_ekle()` — 7 ekle



`onceki ≠ NULL` : başa değil, araya ekliyoruz; baş değişmeyecek.

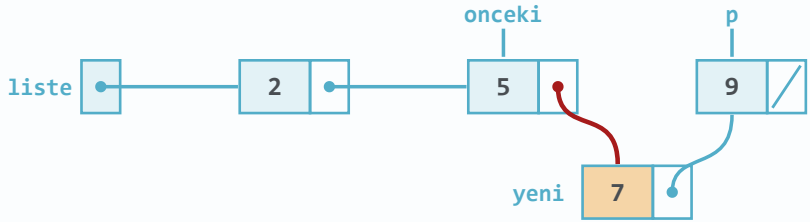
Adım 11 / 12

```
1  void sirali_ekle(oge_gost *bas, oge_gost yeni) {
2      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger < yeni->deger) {
4          onceki = p;
5          p = p->bag;
6      }
7      yeni->bag = p;
▶      if (onceki == NULL) *bas = yeni;  // başa
9      else onceki->bag = yeni;          // araya/sona
10 }
```

DEĞİŞKENLER

- `yeni = &düğüm(7)`
- `onceki = &düğüm(5)`
- `p = &düğüm(9)`
- `*bas = &düğüm(2)`

Kod İzleme: `sirali_ekle()` — 7 ekle



`onceki->bag = yeni` : 5 artık 7'yi gösteriyor. Liste 2 → 5 → 7 → 9.

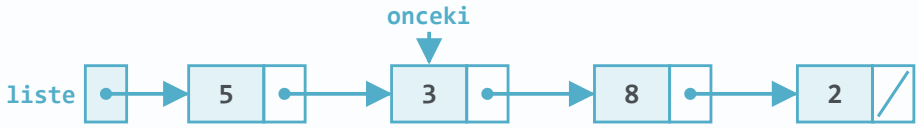
Adım 12 / 12

```
1  void sirali_ekle(oge_gost *bas, oge_gost yeni) {
2      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger < yeni->deger) {
4          onceki = p;
5          p = p->bag;
6      }
7      yeni->bag = p;
8      if (onceki == NULL) *bas = yeni;  // başa
▶      else onceki->bag = yeni;          // araya/sona
10 }
```

DEĞİŞKENLER

- `yeni = &düğüm(7)`
- `onceki = &düğüm(5)`
- `p = &düğüm(9)`
- `*bas = &düğüm(2)`

Silme: silinecek düğümün öncesi gerekir



Tek yönlü bağlı listede bir düğümü silmek için onun **öncesindeki** düğüm gerekir: burada 3.

Adım 1 / 4

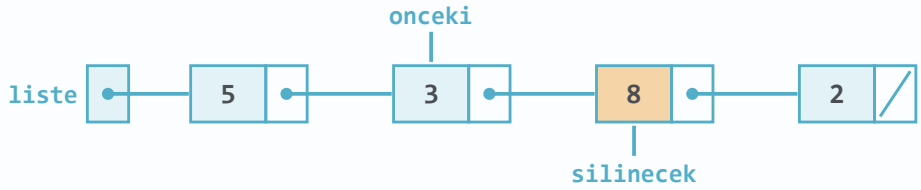
```
1 // onceki'den sonra gelen düğümü listeden ve
2 // bellekten siler
▶ void sonrakini_sil(oge_gost onceki) {
4     oge_gost silinecek = onceki->bag; // adresi tut
5     if (silinecek == NULL) return; // silinecek yok
6     onceki->bag = silinecek->bag;  // bağı düzelt
7     free(silinecek);               // kalıntıyı sil
8 }
```

DEĞİŞKENLER

`onceki = &düğüm(3)`

`silinecek = ?`

Silme: silinecek düğümün öncesi gerekir



`silinecek = onceki→bag` : adres saklandı; yoksa free edemezdik.

Adım 2 / 4

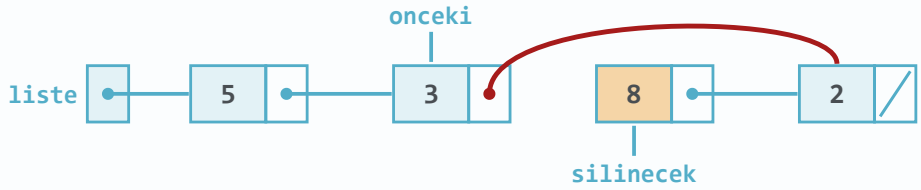
```
1 // onceki'den sonra gelen düğümü listeden ve
2 // bellekten siler
3 void sonrakini_sil(oge_gost onceki) {
▶     oge_gost silinecek = onceki->bag; // adresi tut
5     if (silinecek == NULL) return; // silinecek yok
6     onceki->bag = silinecek->bag;  // bağı düzelt
7     free(silinecek);               // kalıntıyı sil
8 }
```

DEĞİŞKENLER

`onceki = &düğüm(3)`

`silinecek = &düğüm(8)`

Silme: silinecek düğümün öncesi gerekir



`onceki->bag = silinecek->bag` : 3 artık 2'yi gösteriyor, 8 zincirden çıktı.

Adım 3 / 4

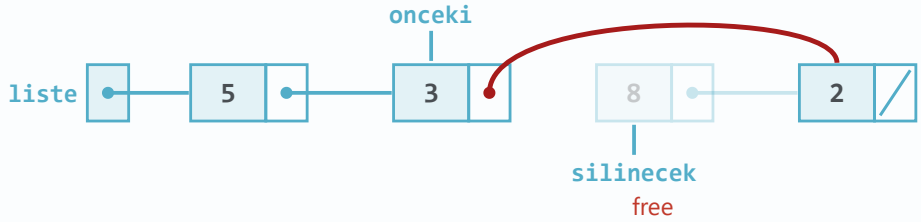
```
1 // onceki'den sonra gelen düğümü listeden ve
2 // bellekten siler
3 void sonrakini_sil(oge_gost onceki) {
4     oge_gost silinecek = onceki->bag; // adresi tut
5     if (silinecek == NULL) return; // silinecek yok
▶     onceki->bag = silinecek->bag;  // bağı düzelt
7     free(silinecek);               // kalıntıyı sil
8 }
```

DEĞİŞKENLER

`onceki = &düğüm(3)`

`silinecek = &düğüm(8)`

Silme: silinecek düğümün öncesi gerekir



`free(silinecek)` : bellekteki kalıntı temizlendi. Baş düğüm silinecekse öncesi yoktur → özel durum (ya da başlık düğümü).

Adım 4 / 4

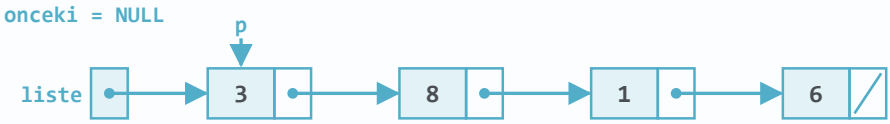
```
1 // onceki'den sonra gelen düğümü listeden ve
2 // bellekten siler
3 void sonrakini_sil(oge_gost onceki) {
4     oge_gost silinecek = onceki->bag; // adresi tut
5     if (silinecek == NULL) return; // silinecek yok
6     onceki->bag = silinecek->bag;  // bağı düzelt
▶     free(silinecek);               // kalıntıyı sil
8 }
```

DEĞİŞKENLER

`onceki = &düğüm(3)`

`silinecek = (serbest)`

Kod İzleme: `sil(&liste, 1)`



Liste 3 → 8 → 1 → 6, silinecek `x = 1`. `p` baştan, `onceki = NULL`.

Adım 1 / 13

```
1  bool sil(oge_gost *bas, int x) {
▶      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger != x) {
4          onceki = p;
5          p = p->bag;
6      }
7      if (p == NULL) return false;         // x yok
8      if (onceki == NULL) *bas = p->bag;   // baştaki
9      else onceki->bag = p->bag;           // atla
10     free(p);
11     return true;
12 }
```

DEĞİŞKENLER

`x = 1` `onceki = NULL` `p = &düğüm(3)`

Kod İzleme: `sil(&liste, 1)`



Koşul `3 ≠ 1 → doğru`. Aramaya devam.

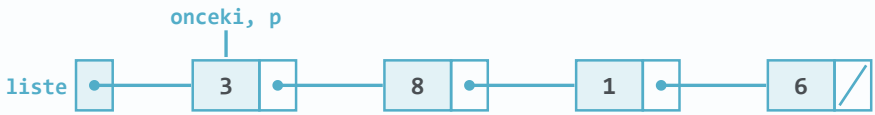
Adım 2 / 13

```
1  bool sil(oge_gost *bas, int x) {
2      oge_gost onceki = NULL, p = *bas;
▶      while (p != NULL && p->deger != x) {
4          onceki = p;
5          p = p->bag;
6      }
7      if (p == NULL) return false;         // x yok
8      if (onceki == NULL) *bas = p->bag;   // baştaki
9      else onceki->bag = p->bag;           // atla
10     free(p);
11     return true;
12 }
```

DEĞİŞKENLER

`x = 1` `onceki = NULL` `p = &düğüm(3)`

Kod İzleme: `sil(&liste, 1)`



`onceki = p` → düğüm(3).

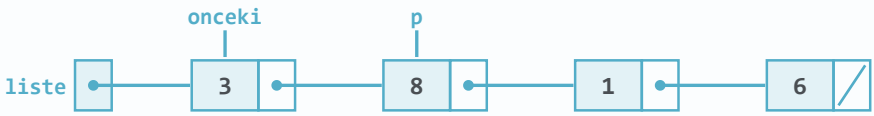
Adım 3 / 13

```
1  bool sil(oge_gost *bas, int x) {
2      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger != x) {
▶          onceki = p;
5          p = p->bag;
6      }
7      if (p == NULL) return false;         // x yok
8      if (onceki == NULL) *bas = p->bag;   // baştaki
9      else onceki->bag = p->bag;           // atla
10     free(p);
11     return true;
12 }
```

DEĞİŞKENLER

- `x = 1`
- `onceki = &düğüm(3)`
- `p = &düğüm(3)`

Kod İzleme: `sil(&liste, 1)`



`p = p→bag` → &düğüm(8).

Adım 4 / 13

```
1  bool sil(oge_gost *bas, int x) {
2      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger != x) {
4          onceki = p;
▶          p = p->bag;
6      }
7      if (p == NULL) return false;         // x yok
8      if (onceki == NULL) *bas = p->bag;   // baştaki
9      else onceki->bag = p->bag;           // atla
10     free(p);
11     return true;
12 }
```

DEĞİŞKENLER

`x = 1` `onceki = &düğüm(3)` `p = &düğüm(8)`

Kod İzleme: `sil(&liste, 1)`



Koşul `8 ≠ 1 → doğru`. Aramaya devam.

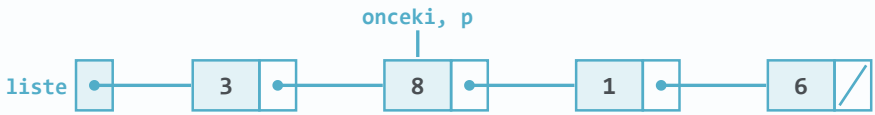
Adım 5 / 13

```
1  bool sil(oge_gost *bas, int x) {
2      oge_gost onceki = NULL, p = *bas;
▶      while (p != NULL && p->deger != x) {
4          onceki = p;
5          p = p->bag;
6      }
7      if (p == NULL) return false;         // x yok
8      if (onceki == NULL) *bas = p->bag;   // baştaki
9      else onceki->bag = p->bag;           // atla
10     free(p);
11     return true;
12 }
```

DEĞİŞKENLER

- `x = 1`
- `onceki = &düğüm(3)`
- `p = &düğüm(8)`

Kod İzleme: `sil(&liste, 1)`



`onceki = p` → düğüm(8).

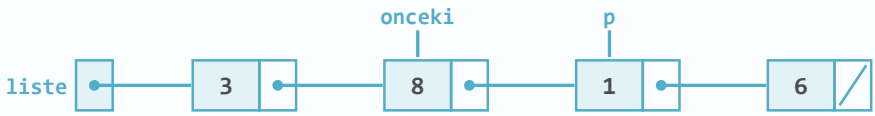
Adım 6 / 13

```
1  bool sil(oge_gost *bas, int x) {
2      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger != x) {
▶          onceki = p;
5          p = p->bag;
6      }
7      if (p == NULL) return false;         // x yok
8      if (onceki == NULL) *bas = p->bag;   // baştaki
9      else onceki->bag = p->bag;           // atla
10     free(p);
11     return true;
12 }
```

DEĞİŞKENLER

- `x = 1`
- `onceki = &düğüm(8)`
- `p = &düğüm(8)`

Kod İzleme: `sil(&liste, 1)`



`p = p→bag` → &düğüm(1).

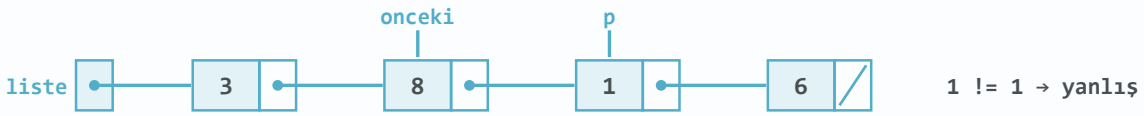
Adım 7 / 13

```
1  bool sil(oge_gost *bas, int x) {
2      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger != x) {
4          onceki = p;
▶          p = p->bag;
6      }
7      if (p == NULL) return false;         // x yok
8      if (onceki == NULL) *bas = p->bag;   // baştaki
9      else onceki->bag = p->bag;           // atla
10     free(p);
11     return true;
12 }
```

DEĞİŞKENLER

`x = 1` `onceki = &düğüm(8)` `p = &düğüm(1)`

Kod İzleme: `sil(&liste, 1)`



Koşul `1 ≠ 1 → yanlış`. Silinecek düğüm bulundu; `onceki` bir gerisinde.

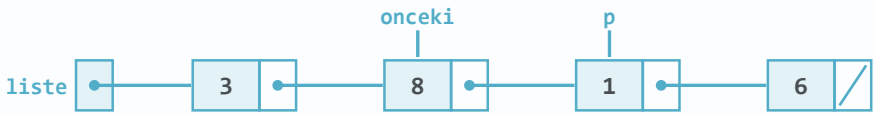
Adım 8 / 13

```
1  bool sil(oge_gost *bas, int x) {
2      oge_gost onceki = NULL, p = *bas;
▶      while (p != NULL && p->deger != x) {
4          onceki = p;
5          p = p->bag;
6      }
7      if (p == NULL) return false;        // x yok
8      if (onceki == NULL) *bas = p->bag;  // baştaki
9      else onceki->bag = p->bag;          // atla
10     free(p);
11     return true;
12 }
```

DEĞİŞKENLER

`x = 1` `onceki = &düğüm(8)` `p = &düğüm(1)`

Kod İzleme: `sil(&liste, 1)`



`p ≠ NULL` : x listede var, silmeye geçilir.

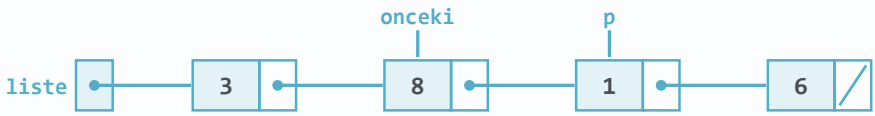
Adım 9 / 13

```
1  bool sil(oge_gost *bas, int x) {
2      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger != x) {
4          onceki = p;
5          p = p->bag;
6      }
▶      if (p == NULL) return false;          // x yok
8      if (onceki == NULL) *bas = p->bag;    // baştaki
9      else onceki->bag = p->bag;            // atla
10     free(p);
11     return true;
12 }
```

DEĞİŞKENLER

`x = 1` `onceki = &düğüm(8)` `p = &düğüm(1)`

Kod İzleme: `sil(&liste, 1)`



`onceki ≠ NULL` : silinen baş düğüm değil; `*bas` değişmeyecek.

Adım 10 / 13

```
1  bool sil(oge_gost *bas, int x) {
2      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger != x) {
4          onceki = p;
5          p = p->bag;
6      }
7      if (p == NULL) return false;        // x yok
▶      if (onceki == NULL) *bas = p->bag;  // baştaki
9      else onceki->bag = p->bag;          // atla
10     free(p);
11     return true;
12 }
```

DEĞİŞKENLER

`x = 1` `onceki = &düğüm(8)` `p = &düğüm(1)`

Kod İzleme: `sil(&liste, 1)`



`onceki→bag = p→bag` : 8 artık 6'yı gösteriyor; 1 zincirden **atlandı**.

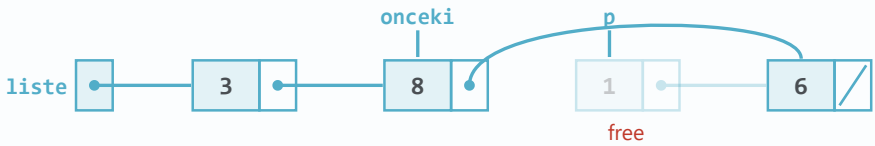
Adım 11 / 13

```
1  bool sil(oge_gost *bas, int x) {
2      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger != x) {
4          onceki = p;
5          p = p->bag;
6      }
7      if (p == NULL) return false;        // x yok
8      if (onceki == NULL) *bas = p->bag;  // baştaki
▶      else onceki->bag = p->bag;          // atla
10     free(p);
11     return true;
12 }
```

DEĞİŞKENLER

`x = 1` `onceki = &düğüm(8)` `p = &düğüm(1)`

Kod İzleme: `sil(&liste, 1)`



`free(p)` : düğüm belleğe iade edildi. Bunu unutursak **bellek sızıntısı** (memory leak) olur.

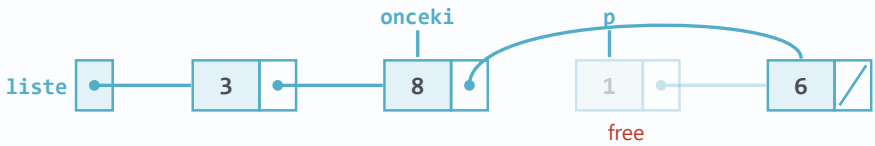
Adım 12 / 13

```
1  bool sil(oge_gost *bas, int x) {
2      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger != x) {
4          onceki = p;
5          p = p->bag;
6      }
7      if (p == NULL) return false;         // x yok
8      if (onceki == NULL) *bas = p->bag;   // baştaki
9      else onceki->bag = p->bag;           // atla
▶      free(p);
11     return true;
12 }
```

DEĞİŞKENLER

`x = 1` `onceki = &düğüm(8)` `p = (serbest)`

Kod İzleme: `sil(&liste, 1)`



Liste 3 → 8 → 6. Arama O(n), bağ değişimi O(1): toplam **O(n)**.

Adım 13 / 13

```
1  bool sil(oge_gost *bas, int x) {
2      oge_gost onceki = NULL, p = *bas;
3      while (p != NULL && p->deger != x) {
4          onceki = p;
5          p = p->bag;
6      }
7      if (p == NULL) return false;         // x yok
8      if (onceki == NULL) *bas = p->bag;   // baştaki
9      else onceki->bag = p->bag;           // atla
10     free(p);
▶      return true;
12 }
```

DEĞİŞKENLER

`x = 1` `onceki = &düğüm(8)` `p = (serbest)`

Arama ve i. düğüme erişim

```
oge_gost ara(oge_gost p, int x) {
    while (p != NULL && p->deger != x)
        p = p->bag;
    return p;                // bulunamazsa NULL
}

oge_gost konum(oge_gost p, int i) {  // 0 tabanlı
    while (p != NULL && i-- > 0)
        p = p->bag;
    return p;                // i >= uzunluk ise NULL
}
```

Kısa devre (short-circuit)

`p ≠ NULL && p→deger ≠ x` : sıra ters olsaydı listenin sonunda NULL'ın içine bakılırdı.

Maliyet

Arama en kötü O(n), `konum(p, i)` O(i). Dizideki `a[i]` gibi O(1) değil.

Sıralı listede

`p→deger > x` görüldüğü an durulabilir; yine de en kötü O(n). İkili arama bağlı listede işe yaramaz.

Sınav sorusu: hangisi doğru?

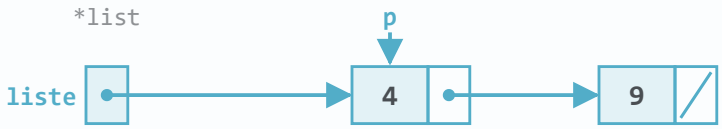
Sınav sorusu. Listeyi teker teker serbest bırakan iki yordam (sol: sık görülen hatalı sürüm, sağ: doğru sürüm). Soldakinde çağıran yordamdaki `list` ne olur?

```
void serbest_birak( list_gost *list ){
    oge_gost p = *list;
    if( *list ){
        p = *list->bag;
        free(*list);
        serbest_birak(&p);
}}
```

```
void serbest_birak( list_gost *list ){
    oge_gost p = *list;
    if( p ){
        *list = p->bag;
        free(p);
        serbest_birak(list);
}}
```

▶ **Çözüm**

Kod İzleme: `serbest_birak(&liste)`



1. çağrı: `p = *list` → &düğüm(4).

Adım 1 / 15

```
1 void serbest_birak(oge_gost *list) {
▶     oge_gost p = *list;
3     if (p) {
4         *list = p->bag;      // önce başı ilerlet
5         free(p);             // sonra eski başı bırak
6         serbest_birak(list); // aynı adresle devam
7     }
8 }
```

DEĞİŞKENLER

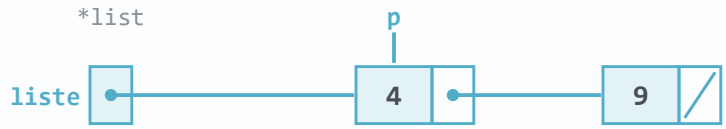
`list = &liste` `*list = &düğüm(4)` `p = &düğüm(4)`

ÇAĞRI YIĞINI

`serbest_birak(&liste)`

`main()`

Kod İzleme: `serbest_birak(&liste)`



`p ≠ NULL` : bırakılacak düğüm var.

Adım 2 / 15

```
1 void serbest_birak(oge_gost *list) {
2     oge_gost p = *list;
▶     if (p) {
4         *list = p->bag;       // önce başı ilerlet
5         free(p);              // sonra eski başı bırak
6         serbest_birak(list); // aynı adresle devam
7     }
8 }
```

DEĞİŞKENLER

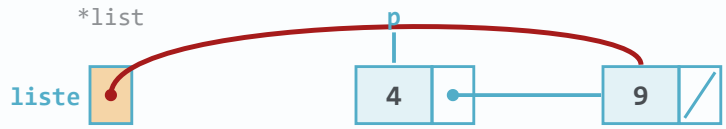
`list = &liste` `*list = &düğüm(4)` `p = &düğüm(4)`

ÇAĞRI YIĞINI

`serbest_birak(&liste)`

`main()`

Kod İzleme: `serbest_birak(&liste)`



`*list = p→bag` : main'deki `liste` artık &düğüm(9).

Adım 3 / 15

```
1 void serbest_birak(oge_gost *list) {
2     oge_gost p = *list;
3     if (p) {
▶         *list = p->bag;       // önce başı ilerlet
5         free(p);              // sonra eski başı bırak
6         serbest_birak(list); // aynı adresle devam
7     }
8 }
```

DEĞİŞKENLER

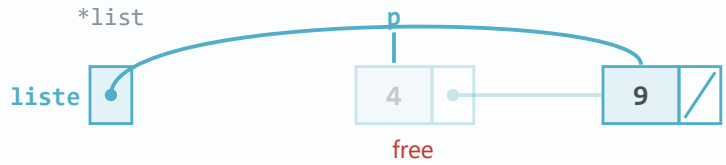
`list = &liste` `*list = &düğüm(9)` `p = &düğüm(4)`

ÇAĞRI YIĞINI

`serbest_birak(&liste)`

`main()`

Kod İzleme: `serbest_birak(&liste)`



`free(p)` : düğüm(4) iade edildi; `p` artık sarkan (dangling) işaretçi, kullanılmaz.

Adım 4 / 15

```
1 void serbest_birak(oge_gost *list) {
2     oge_gost p = *list;
3     if (p) {
4         *list = p->bag;       // önce başı ilerlet
▶         free(p);              // sonra eski başı bırak
6         serbest_birak(list); // aynı adresle devam
7     }
8 }
```

DEĞİŞKENLER

`list = &liste` `*list = &düğüm(9)`

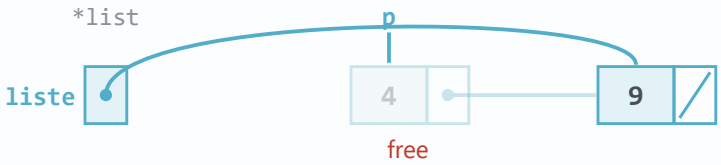
`p = &düğüm(4) (serbest)`

ÇAĞRI YIĞINI

`serbest_birak(&liste)`

`main()`

Kod İzleme: `serbest_birak(&liste)`



Aynı `list` adresiyle özyinelemeli çağrı: kalan liste bırakılacak.

Adım 5 / 15

```
1 void serbest_birak(oge_gost *list) {
2     oge_gost p = *list;
3     if (p) {
4         *list = p->bag;      // önce başı ilerlet
5         free(p);             // sonra eski başı bırak
▶         serbest_birak(list); // aynı adresle devam
7     }
8 }
```

DEĞİŞKENLER

`list = &liste` `*list = &düğüm(9)` `p = &düğüm(4) (serbest)`

ÇAĞRI YIĞINI

`serbest_birak(&liste)`

`main()`

Kod İzleme: `serbest_birak(&liste)`



2. çağrı: `p = *list` → &düğüm(9).

Adım 6 / 15

```
1 void serbest_birak(oge_gost *list) {
▶       oge_gost p = *list;
3       if (p) {
4           *list = p->bag;      // önce başı ilerlet
5           free(p);             // sonra eski başı bırak
6           serbest_birak(list); // aynı adresle devam
7       }
8   }
```

DEĞİŞKENLER

`list = &liste` `*list = &düğüm(9)` `p = &düğüm(9)`

ÇAĞRI YIĞINI

- `serbest_birak(&liste)`
- `serbest_birak(&liste)`
- `main()`

Kod İzleme: `serbest_birak(&liste)`



`p ≠ NULL` : bırakılacak düğüm var.

Adım 7 / 15

```
1 void serbest_birak(oge_gost *list) {
2     oge_gost p = *list;
▶     if (p) {
4         *list = p->bag;       // önce başı ilerlet
5         free(p);              // sonra eski başı bırak
6         serbest_birak(list); // aynı adresle devam
7     }
8 }
```

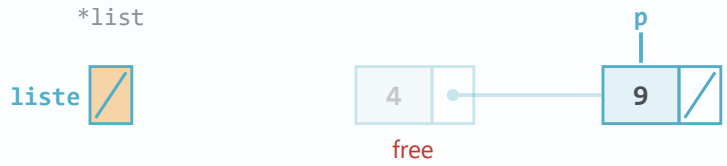
DEĞİŞKENLER

`list = &liste` `*list = &düğüm(9)` `p = &düğüm(9)`

ÇAĞRI YIĞINI

- `serbest_birak(&liste)`
- `serbest_birak(&liste)`
- `main()`

Kod İzleme: `serbest_birak(&liste)`



`*list = p->bag` : main'deki `liste` artık NULL.

Adım 8 / 15

```
1 void serbest_birak(oge_gost *list) {
2     oge_gost p = *list;
3     if (p) {
▶         *list = p->bag;       // önce başı ilerlet
5         free(p);              // sonra eski başı bırak
6         serbest_birak(list); // aynı adresle devam
7     }
8 }
```

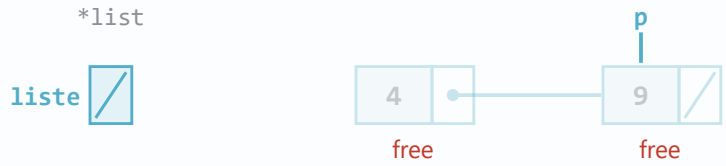
DEĞİŞKENLER

`list = &liste` `*list = NULL` `p = &düğüm(9)`

ÇAĞRI YIĞINI

- `serbest_birak(&liste)`
- `serbest_birak(&liste)`
- `main()`

Kod İzleme: `serbest_birak(&liste)`



`free(p)` : düğüm(9) iade edildi; `p` artık sarkan (dangling) işaretçi, kullanılmaz.

Adım 9 / 15

```
1 void serbest_birak(oge_gost *list) {
2     oge_gost p = *list;
3     if (p) {
4         *list = p->bag;       // önce başı ilerlet
▶         free(p);              // sonra eski başı bırak
6         serbest_birak(list); // aynı adresle devam
7     }
8 }
```

DEĞİŞKENLER

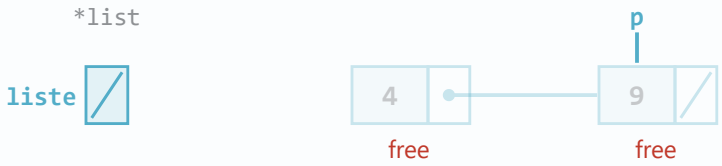
`list = &liste` `*list = NULL`

`p = &düğüm(9) (serbest)`

ÇAĞRI YIĞINI

- `serbest_birak(&liste)`
- `serbest_birak(&liste)`
- `main()`

Kod İzleme: `serbest_birak(&liste)`



Aynı `list` adresiyle özyinelemeli çağrı: kalan liste bırakılacak.

Adım 10 / 15

```
1 void serbest_birak(oge_gost *list) {
2     oge_gost p = *list;
3     if (p) {
4         *list = p->bag;      // önce başı ilerlet
5         free(p);             // sonra eski başı bırak
▶         serbest_birak(list); // aynı adresle devam
7     }
8 }
```

DEĞİŞKENLER

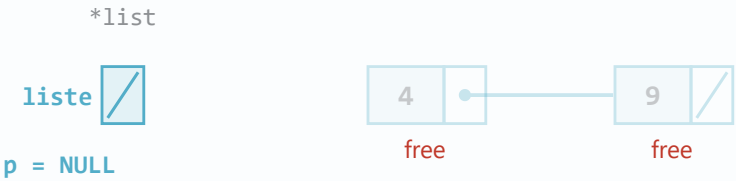
`list = &liste` `*list = NULL`

`p = &düğüm(9) (serbest)`

ÇAĞRI YIĞINI

- `serbest_birak(&liste)`
- `serbest_birak(&liste)`
- `main()`

Kod İzleme: `serbest_birak(&liste)`



3. çağrı: `p = *list` → NULL.

Adım 11 / 15

```
1 void serbest_birak(oge_gost *list) {
▶     oge_gost p = *list;
3     if (p) {
4         *list = p->bag;      // önce başı ilerlet
5         free(p);             // sonra eski başı bırak
6         serbest_birak(list); // aynı adresle devam
7     }
8 }
```

DEĞİŞKENLER

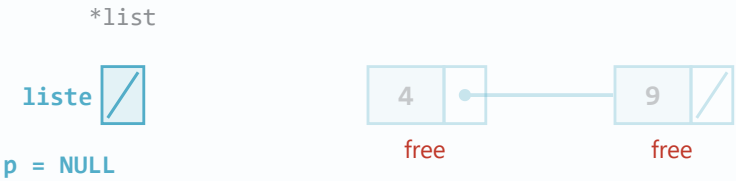
`list = &liste` `*list = NULL`

`p = NULL`

ÇAĞRI YIĞINI

- **serbest_birak(&liste)**
- serbest_birak(&liste)
- serbest_birak(&liste)
- main()

Kod İzleme: `serbest_birak(&liste)`



`p == NULL` : liste bitti, çağrı hiçbir şey yapmadan döner. **Taban durumu.**

Adım 12 / 15

```
1 void serbest_birak(oge_gost *list) {
2     oge_gost p = *list;
▶     if (p) {
4         *list = p->bag;       // önce başı ilerlet
5         free(p);              // sonra eski başı bırak
6         serbest_birak(list); // aynı adresle devam
7     }
8 }
```

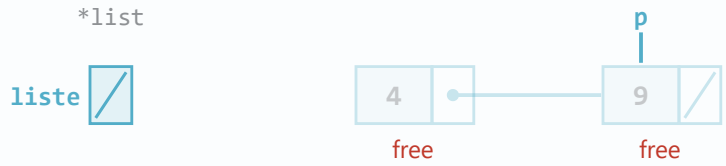
DEĞİŞKENLER

`list = &liste` `*list = NULL` `p = NULL`

ÇAĞRI YIĞINI

- **serbest_birak(&liste)**
- serbest_birak(&liste)
- serbest_birak(&liste)
- main()

Kod İzleme: `serbest_birak(&liste)`



2. çağrıya dönüldü; yapılacak iş kalmadı.

Adım 13 / 15

```
1 void serbest_birak(oge_gost *list) {
2     oge_gost p = *list;
3     if (p) {
4         *list = p->bag;      // önce başı ilerlet
5         free(p);             // sonra eski başı bırak
6         serbest_birak(list); // aynı adresle devam
▶     }
8 }
```

DEĞİŞKENLER

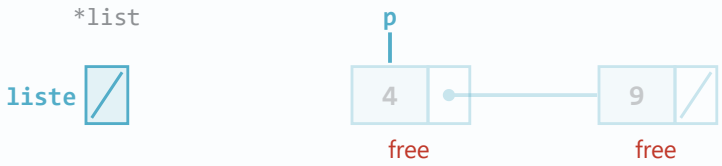
`list = &liste` `*list = NULL`

`p = &düğüm(9) (serbest)`

ÇAĞRI YIĞINI

- `serbest_birak(&liste)`
- `serbest_birak(&liste)`
- `main()`

Kod İzleme: `serbest_birak(&liste)`



1. çağrıya dönüldü; yapılacak iş kalmadı.

Adım 14 / 15

```
1 void serbest_birak(oge_gost *list) {
2     oge_gost p = *list;
3     if (p) {
4         *list = p->bag;      // önce başı ilerlet
5         free(p);             // sonra eski başı bırak
6         serbest_birak(list); // aynı adresle devam
▶     }
8 }
```

DEĞİŞKENLER

`list = &liste` `*list = NULL`

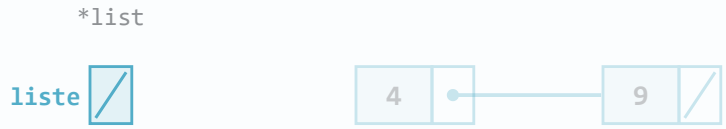
`p = &düğüm(4) (serbest)`

ÇAĞRI YIĞINI

`serbest_birak(&liste)`

`main()`

Kod İzleme: `serbest_birak(&liste)`



Tüm düğümler bırakıldı ve main'deki `liste == NULL` : sarkan işaretçi kalmadı.

Adım 15 / 15

```
1 void serbest_birak(oge_gost *list) {
2     oge_gost p = *list;
3     if (p) {
4         *list = p->bag;      // önce başı ilerlet
5         free(p);             // sonra eski başı bırak
6         serbest_birak(list); // aynı adresle devam
▶     }
8 }
```

DEĞİŞKENLER

`liste = NULL`

ÇAĞRI YIĞINI

`main()`

Aynı işin yinelemeli hâli

```
void liste_bosalt(oge_gost *list) {
    while (*list != NULL) {
        oge_gost p = *list;
        *list = p->bag;      // önce ilerle
        free(p);             // sonra bırak
    }
}
```

Özyinelemeli sürüm

Her düğüm için bir çağrı çerçevesi: n = 10^6 düğümde yığın taşabilir (stack overflow).

Yinelemeli sürüm

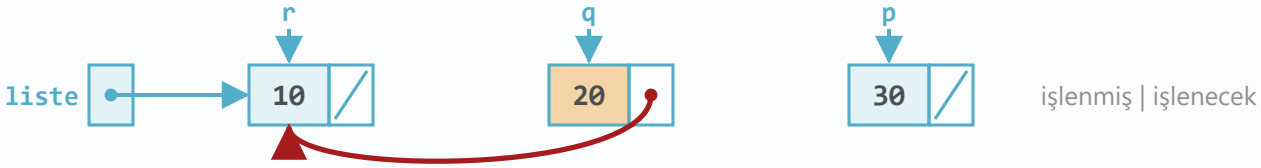
O(n) zaman, **O(1)** ek bellek; `*list` sonunda NULL.

Ortak kural

free'den **önce** sonraki adres alınır; free edilmiş düğümün `bag` alanı okunmaz.

Soru: bağları tersine çevir (`liste_devrik_al`)

SORU. `list` işaretçisiyle işaret edilen sıralı listenin bağ alanlarını tersine çeviren `liste_devrik_al` yordamını yazınız.



Ara durum: r ve q arası çevrildi, p'den sonrası henüz eski yönde.

Üç işaretçi

`r` en geride, `q` ortada, `p` en ileride; her turda bir bağ çevrilir.

Yerinde (in-place)

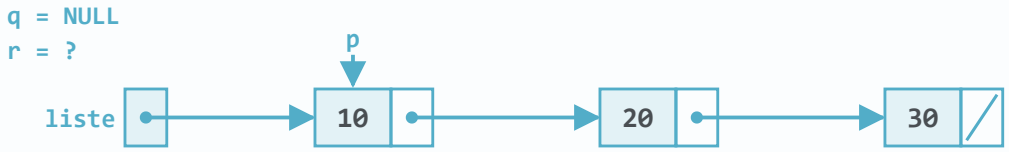
Yeni düğüm yok, kopya yok: O(n) zaman, O(1) ek bellek.

Baş değişir

Bu yüzden liste `oge_gost *list` ile adresle gönderilir.

Hocadan "Oge_ekle, oge_cikar, malloc, ekstra bellek, ekstra işler yok; hiçbir şey yok!"

Kod İzleme: `liste_devrik_al(&liste)`



Üç işaretçi: `p` en ileride (baş), `q` ortada (NULL), `r` en geride (henüz değer yok).

Adım 1 / 18

```
1  void liste_devrik_al(oge_gost *list) {
▶      oge_gost r, p = *list, q = NULL;
3      while (p) {
4          r = q;          // r, q'yu takip eder
5          q = p;          // q, p'yi takip eder
6          p = p->bag;     // p'yi ilerlet
7          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

`r = ?` `q = NULL` `p = &düğüm(10)` `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



`p ≠ NULL` : tur başlar.

Adım 2 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
▶      while (p) {
4          r = q;          // r, q'yu takip eder
5          q = p;          // q, p'yi takip eder
6          p = p->bag;     // p'yi ilerlet
7          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

`r = ?` `q = NULL` `p = &düğüm(10)` `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



1. tur: `r = q` → NULL.

Adım 3 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
3      while (p) {
▶          r = q;          // r, q'yu takip eder
5          q = p;          // q, p'yi takip eder
6          p = p->bag;     // p'yi ilerlet
7          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

- `r = NULL`
- `q = NULL`
- `p = &düğüm(10)`
- `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



`q = p` → &düğüm(10): bu turda bağı çevrilecek düğüm.

Adım 4 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
3      while (p) {
4          r = q;          // r, q'yu takip eder
▶          q = p;          // q, p'yi takip eder
6          p = p->bag;     // p'yi ilerlet
7          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

`r = NULL` `q = &düğüm(10)` `p = &düğüm(10)` `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



`p = p→bag` → &düğüm(20). **Önce** ilerliyoruz ki kalan liste kaybolmasın.

Adım 5 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
3      while (p) {
4          r = q;          // r, q'yu takip eder
5          q = p;          // q, p'yi takip eder
▶          p = p->bag;     // p'yi ilerlet
7          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

- `r = NULL`
- `q = &düğüm(10)`
- `p = &düğüm(20)`
- `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



`q→bag = r` : düğüm(10) artık NULL'ı gösteriyor.

Adım 6 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
3      while (p) {
4          r = q;          // r, q'yu takip eder
5          q = p;          // q, p'yi takip eder
6          p = p->bag;     // p'yi ilerlet
▶          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

- `r = NULL`
- `q = &düğüm(10)`
- `p = &düğüm(20)`
- `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



`p ≠ NULL` : tur başlar.

Adım 7 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
▶      while (p) {
4          r = q;          // r, q'yu takip eder
5          q = p;          // q, p'yi takip eder
6          p = p->bag;     // p'yi ilerlet
7          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

`r = NULL` `q = &düğüm(10)` `p = &düğüm(20)` `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



2. tur: `r = q` → &düğüm(10).

Adım 8 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
3      while (p) {
▶          r = q;          // r, q'yu takip eder
5          q = p;          // q, p'yi takip eder
6          p = p->bag;     // p'yi ilerlet
7          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

- `r = &düğüm(10)`
- `q = &düğüm(10)`
- `p = &düğüm(20)`
- `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



`q = p` → &düğüm(20): bu turda bağı çevrilecek düğüm.

Adım 9 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
3      while (p) {
4          r = q;          // r, q'yu takip eder
▶          q = p;          // q, p'yi takip eder
6          p = p->bag;     // p'yi ilerlet
7          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

`r = &düğüm(10)` `q = &düğüm(20)` `p = &düğüm(20)` `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



`p = p→bag` → &düğüm(30). **Önce** ilerliyoruz ki kalan liste kaybolmasın.

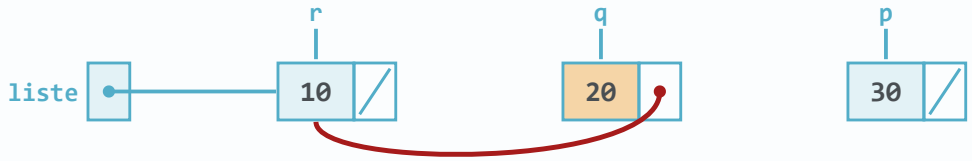
Adım 10 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
3      while (p) {
4          r = q;          // r, q'yu takip eder
5          q = p;          // q, p'yi takip eder
▶          p = p->bag;     // p'yi ilerlet
7          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

`r = &düğüm(10)` `q = &düğüm(20)` `p = &düğüm(30)` `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



`q→bag = r` : düğüm(20) artık düğüm(10)'u gösteriyor.

Adım 11 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
3      while (p) {
4          r = q;          // r, q'yu takip eder
5          q = p;          // q, p'yi takip eder
6          p = p->bag;     // p'yi ilerlet
▶          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

- `r = &düğüm(10)`
- `q = &düğüm(20)`
- `p = &düğüm(30)`
- `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



`p ≠ NULL` : tur başlar.

Adım 12 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
▶      while (p) {
4          r = q;          // r, q'yu takip eder
5          q = p;          // q, p'yi takip eder
6          p = p->bag;     // p'yi ilerlet
7          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

- `r = &düğüm(10)`
- `q = &düğüm(20)`
- `p = &düğüm(30)`
- `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



3. tur: `r = q` → &düğüm(20).

Adım 13 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
3      while (p) {
▶          r = q;          // r, q'yu takip eder
5          q = p;          // q, p'yi takip eder
6          p = p->bag;     // p'yi ilerlet
7          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

`r = &düğüm(20)` `q = &düğüm(20)` `p = &düğüm(30)` `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



`q = p` → &düğüm(30): bu turda bağı çevrilecek düğüm.

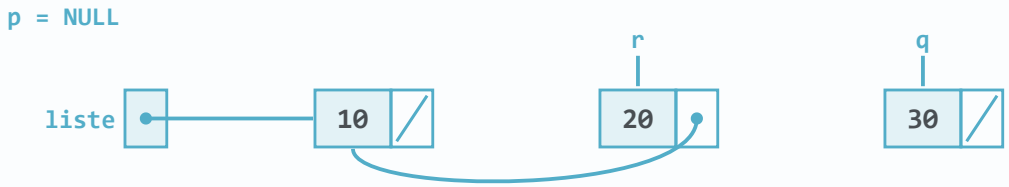
Adım 14 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
3      while (p) {
4          r = q;          // r, q'yu takip eder
▶          q = p;          // q, p'yi takip eder
6          p = p->bag;     // p'yi ilerlet
7          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

- `r = &düğüm(20)`
- `q = &düğüm(30)`
- `p = &düğüm(30)`
- `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



`p = p→bag` → NULL. **Önce** ilerliyoruz ki kalan liste kaybolmasın.

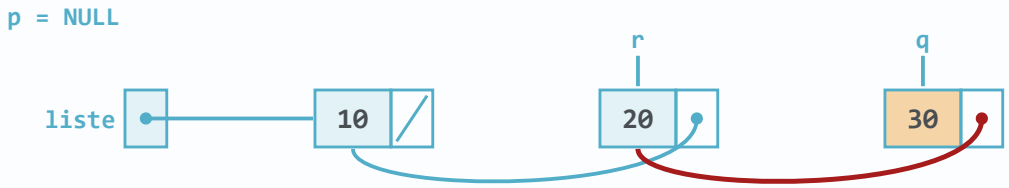
Adım 15 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
3      while (p) {
4          r = q;          // r, q'yu takip eder
5          q = p;          // q, p'yi takip eder
▶          p = p->bag;     // p'yi ilerlet
7          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

`r = &düğüm(20)` `q = &düğüm(30)` `p = NULL` `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



`q→bag = r` : düğüm(30) artık düğüm(20)'yi gösteriyor.

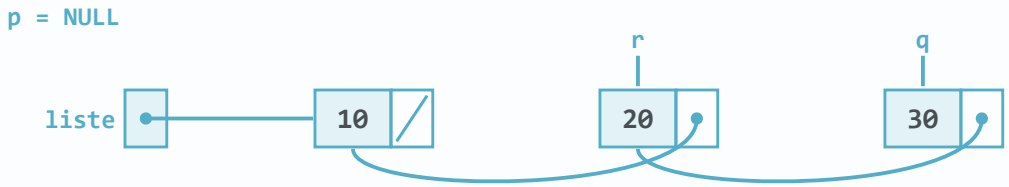
Adım 16 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
3      while (p) {
4          r = q;          // r, q'yu takip eder
5          q = p;          // q, p'yi takip eder
6          p = p->bag;     // p'yi ilerlet
▶          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

`r = &düğüm(20)` `q = &düğüm(30)` `p = NULL` `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



`p == NULL` : döngü biter; q son düğümde.

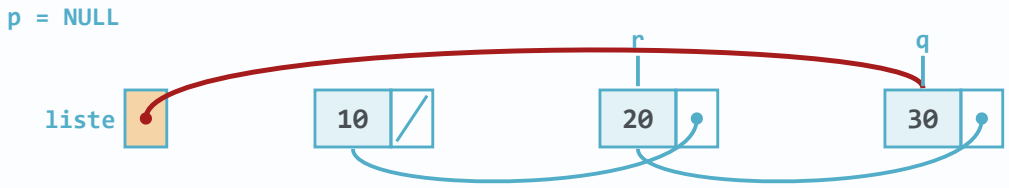
Adım 17 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
▶      while (p) {
4          r = q;          // r, q'yu takip eder
5          q = p;          // q, p'yi takip eder
6          p = p->bag;     // p'yi ilerlet
7          q->bag = r;     // bağı tersine çevir
8      }
9      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

`r = &düğüm(20)` `q = &düğüm(30)` `p = NULL` `*list = &düğüm(10)`

Kod İzleme: `liste_devrik_al(&liste)`



`*list = q` : eski son düğüm yeni baş. 30 → 20 → 10; **O(n)** zaman, **O(1)** ek bellek, malloc yok.

Adım 18 / 18

```
1  void liste_devrik_al(oge_gost *list) {
2      oge_gost r, p = *list, q = NULL;
3      while (p) {
4          r = q;          // r, q'yu takip eder
5          q = p;          // q, p'yi takip eder
6          p = p->bag;     // p'yi ilerlet
7          q->bag = r;     // bağı tersine çevir
8      }
▶      *list = q;          // son öğe liste başı
10 }
```

DEĞİŞKENLER

`r = &düğüm(20)` `q = &düğüm(30)` `p = NULL` `*list = &düğüm(30)`

İki listeyi uç uca birleştirme (concatenate)



L1 = 1 → 4, L2 = 6 → 9. İkisi de boş değil.

Adım 1 / 4

```
1  // L1'in sonuna L2'yi bağlar, başı döndürür
2  oge_gost birlestir(oge_gost L1, oge_gost L2) {
3      oge_gost p;
▶      if (L1 == NULL) return L2;
▶      if (L2 != NULL) {
6          for (p = L1; p->bag != NULL; p = p->bag)
7              ;                    // L1'in son düğümü
8          p->bag = L2;
9      }
10     return L1;
11 }
```

DEĞİŞKENLER

`L1 = &düğüm(1)` `L2 = &düğüm(6)` `p = ?`

İki listeyi uç uca birleştirme (concatenate)



`p` L1'in başından yürür.


```
1  // L1'in sonuna L2'yi bağlar, başı döndürür
2  oge_gost birlestir(oge_gost L1, oge_gost L2) {
3      oge_gost p;
4      if (L1 == NULL) return L2;
5      if (L2 != NULL) {
▶          for (p = L1; p->bag != NULL; p = p->bag)
7              ;                    // L1'in son düğümü
8          p->bag = L2;
9      }
10     return L1;
11 }
```

DEĞİŞKENLER

`L1 = &düğüm(1)` `L2 = &düğüm(6)`

`p = &düğüm(1)`

İki listeyi uç uca birleştirme (concatenate)



`p→bag == NULL` : L1'in son düğümü bulundu. Maliyet **O(|L1|)**.


```
1  // L1'in sonuna L2'yi bağlar, başı döndürür
2  oge_gost birlestir(oge_gost L1, oge_gost L2) {
3      oge_gost p;
4      if (L1 == NULL) return L2;
5      if (L2 != NULL) {
▶          for (p = L1; p->bag != NULL; p = p->bag)
7              ;                    // L1'in son düğümü
8          p->bag = L2;
9      }
10     return L1;
11 }
```

DEĞİŞKENLER

`L1 = &düğüm(1)` `L2 = &düğüm(6)`

`p = &düğüm(4)`

İki listeyi uç uca birleştirme (concatenate)



`p→bag = L2` : 1 → 4 → 6 → 9. Kopya yok. Dairesel listede `son` işaretçisiyle bu iş O(1) olur.

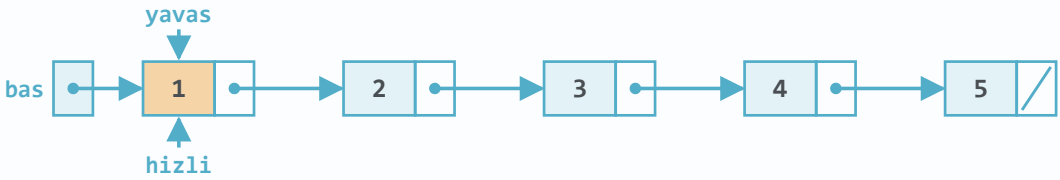
Adım 4 / 4

```
1  // L1'in sonuna L2'yi bağlar, başı döndürür
2  oge_gost birlestir(oge_gost L1, oge_gost L2) {
3      oge_gost p;
4      if (L1 == NULL) return L2;
5      if (L2 != NULL) {
6          for (p = L1; p->bag != NULL; p = p->bag)
7              ;                    // L1'in son düğümü
▶          p->bag = L2;
9      }
10     return L1;
11 }
```

DEĞİŞKENLER

`L1 = &düğüm(1)` `L2 = &düğüm(6)` `p = &düğüm(4)`

Orta düğüm: yavaş ve hızlı işaretçi



İkisi de baştan başlar.

Adım 1 / 4

```
1 oge_gost orta(oge_gost bas) {
▶     oge_gost yavas = bas, hizli = bas;
3     while (hizli != NULL && hizli->bag != NULL) {
4         yavas = yavas->bag;        // 1 adım
5         hizli = hizli->bag->bag;   // 2 adım
6     }
7     return yavas;
8 }
```

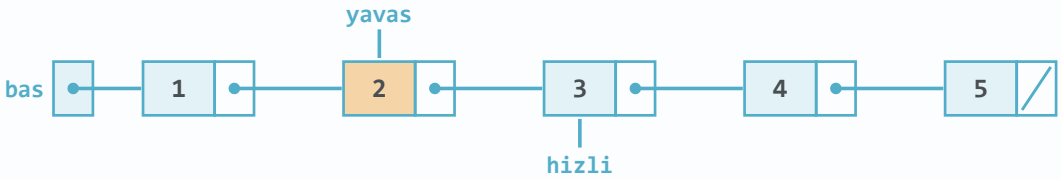
DEĞİŞKENLER

`yavas = &düğüm(1)`

`hizli = &düğüm(1)`

Çift uzunlukta (1 → 2 → 3 → 4) ikinci orta (3) döner; ilk orta için koşul `hizli→bag→bag ≠ NULL` olarak kurulur.

Orta düğüm: yavaş ve hızlı işaretçi



yavas 1, hizli 2 adım atar.

Adım 2 / 4

```
1 oge_gost orta(oge_gost bas) {
2     oge_gost yavas = bas, hizli = bas;
3     while (hizli != NULL && hizli->bag != NULL) {
          yavas = yavas->bag;        // 1 adım
          hizli = hizli->bag->bag;   // 2 adım
6     }
7     return yavas;
8 }
```

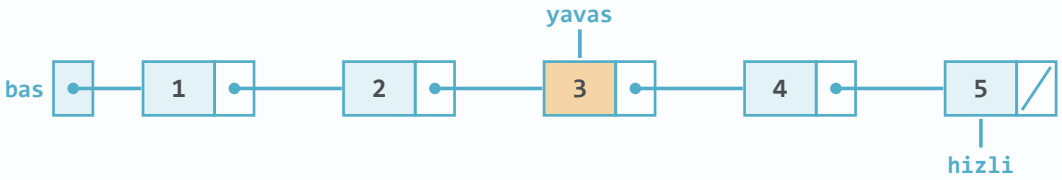
DEĞİŞKENLER

`yavas = &düğüm(2)`

`hizli = &düğüm(3)`

Çift uzunlukta (1 → 2 → 3 → 4) ikinci orta (3) döner; ilk orta için koşul `hizli→bag→bag ≠ NULL` olarak kurulur.

Orta düğüm: yavaş ve hızlı işaretçi



hizli son düğümde (`hizli→bag == NULL`): döngü biter, **yavas = 3** ortadadır.

Adım 3 / 4

```
1 oge_gost orta(oge_gost bas) {
2     oge_gost yavas = bas, hizli = bas;
3     while (hizli != NULL && hizli->bag != NULL) {
          yavas = yavas->bag;        // 1 adım
          hizli = hizli->bag->bag;   // 2 adım
6     }
7     return yavas;
8 }
```

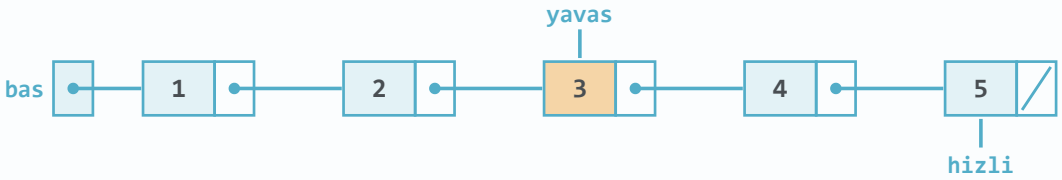
DEĞİŞKENLER

`yavas = &düğüm(3)`

`hizli = &düğüm(5)`

Çift uzunlukta (1 → 2 → 3 → 4) ikinci orta (3) döner; ilk orta için koşul `hizli→bag→bag ≠ NULL` olarak kurulur.

Orta düğüm: yavaş ve hızlı işaretçi



`return yavas` : tek geçiş, O(n) zaman, O(1) ek bellek.

Adım 4 / 4

```
1 oge_gost orta(oge_gost bas) {
2     oge_gost yavas = bas, hizli = bas;
3     while (hizli != NULL && hizli->bag != NULL) {
4         yavas = yavas->bag;         // 1 adım
5         hizli = hizli->bag->bag;    // 2 adım
6     }
▶     return yavas;
8 }
```

DEĞİŞKENLER

`yavas = &düğüm(3)`

`hizli = &düğüm(5)`

Çift uzunlukta (1 → 2 → 3 → 4) ikinci orta (3) döner; ilk orta için koşul `hizli→bag→bag ≠ NULL` olarak kurulur.

KLASİK YORDAMLAR

Floyd'un döngü tespiti algoritması (tavşan ve kaplumbağa)

Sorun

Hatalı bir bağ (`son→bag = ortadaki`) listeyi döngüye sokar; `yazdir` hiç bitmez.

Fikir

yavas 1, hizli 2 adım atar. Döngü yoksa hizli NULL'a varır; varsa döngüye girince aradaki uzaklık her tur 1 azalır → mutlaka buluşurlar.

Maliyet

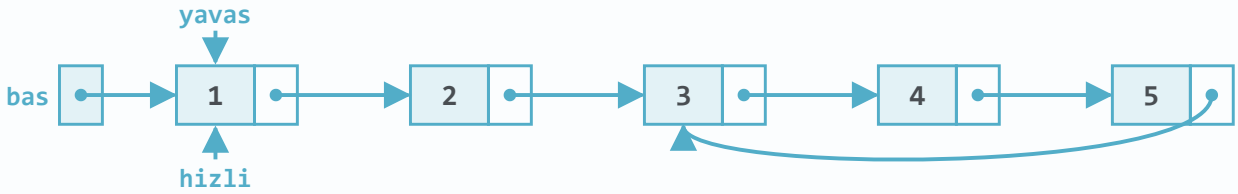
O(n) zaman, **O(1)** ek bellek. Ziyaret edilenleri bir kümede tutmak da çalışır ama O(n) bellek ister.

Döngünün başı

Buluşmadan sonra bir işaretçi başa alınır; ikisi birer adım ilerletilir: yeniden buluştukları düğüm döngünün girişidir.

Floyd (1967); Knuth, TAOCP Cilt 2, Alıştırma 3.1-6.

Kod İzleme: `dongu_var()` — tavşan ve kaplumbağa



5 → 3: döngü

Kaplumbağa (`yavas`) ve tavşan (`hizli`) aynı yerden, baştan başlar. Listede 5 → 3 döngüsü var.

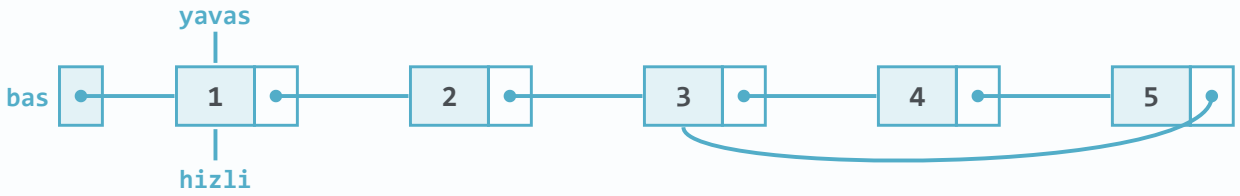
Adım 1 / 13

```
1 bool dongu_var(oge_gost bas) {
▶       oge_gost yavas = bas, hizli = bas;
3       while (hizli != NULL && hizli->bag != NULL) {
4           yavas = yavas->bag;          // 1 adım
5           hizli = hizli->bag->bag;     // 2 adım
6           if (yavas == hizli) return true;
7       }
8       return false;
9   }
```

DEĞİŞKENLER

- `yavas = &düğüm(1)`
- `hizli = &düğüm(1)`

Kod İzleme: `dongu_var()` — tavşan ve kaplumbağa



5 → 3: döngü

`hizli` ve sonraki NULL değil: koşmaya devam.

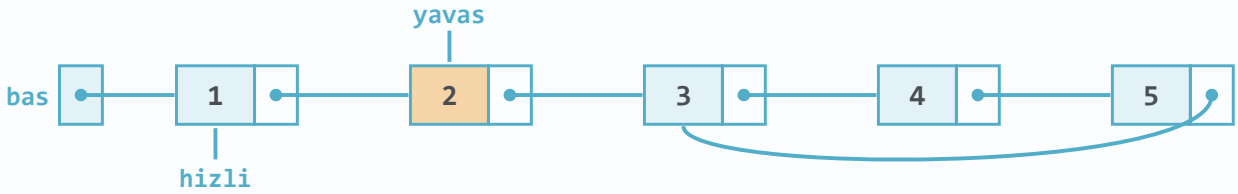
Adım 2 / 13

```
1 bool dongu_var(oge_gost bas) {
2     oge_gost yavas = bas, hizli = bas;
▶     while (hizli != NULL && hizli->bag != NULL) {
4         yavas = yavas->bag;          // 1 adım
5         hizli = hizli->bag->bag;     // 2 adım
6         if (yavas == hizli) return true;
7     }
8     return false;
9 }
```

DEĞİŞKENLER

- `yavas = &düğüm(1)`
- `hizli = &düğüm(1)`

Kod İzleme: `dongu_var()` — tavşan ve kaplumbağa



5 → 3: döngü

1. tur: kaplumbağa 1 adım → düğüm(2).

Adım 3 / 13

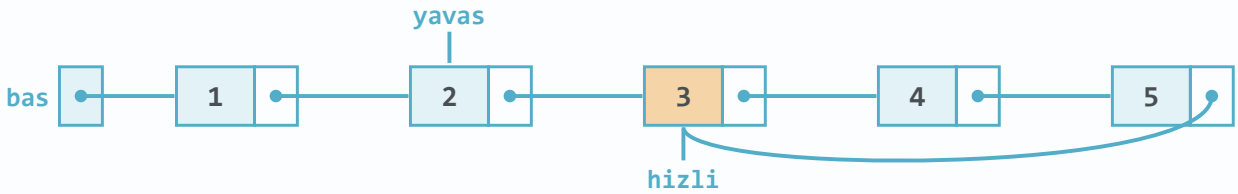
```
1 bool dongu_var(oge_gost bas) {
2     oge_gost yavas = bas, hizli = bas;
3     while (hizli != NULL && hizli->bag != NULL) {
▶         yavas = yavas->bag;          // 1 adım
5         hizli = hizli->bag->bag;     // 2 adım
6         if (yavas == hizli) return true;
7     }
8     return false;
9 }
```

DEĞİŞKENLER

`yavas = &düğüm(2)`

`hizli = &düğüm(1)`

Kod İzleme: `dongu_var()` — tavşan ve kaplumbağa



5 → 3: döngü

Tavşan 2 adım → düğüm(3).

Adım 4 / 13

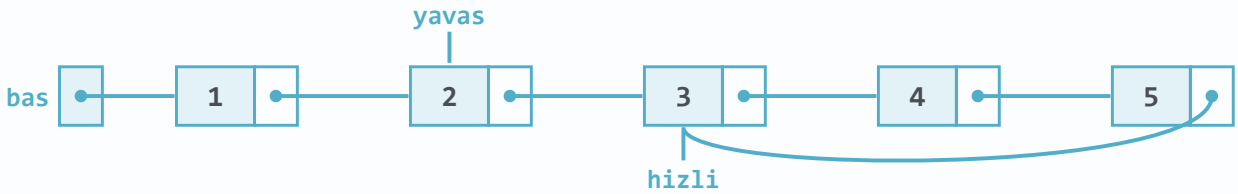
```
1 bool dongu_var(oge_gost bas) {
2     oge_gost yavas = bas, hizli = bas;
3     while (hizli != NULL && hizli->bag != NULL) {
4         yavas = yavas->bag;          // 1 adım
▶         hizli = hizli->bag->bag;     // 2 adım
6         if (yavas == hizli) return true;
7     }
8     return false;
9 }
```

DEĞİŞKENLER

`yavas = &düğüm(2)`

`hizli = &düğüm(3)`

Kod İzleme: `dongu_var()` — tavşan ve kaplumbağa



5 → 3: döngü

Aynı düğümde değiller; devam.

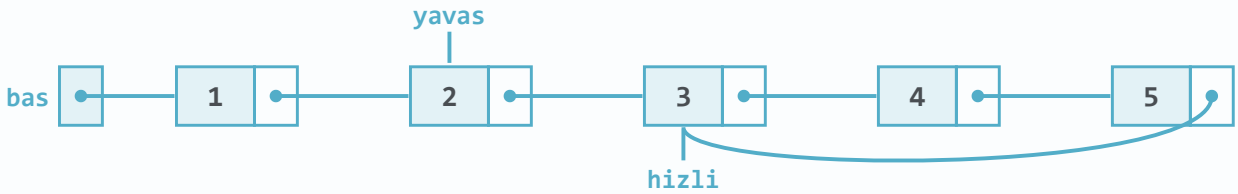
Adım 5 / 13

```
1 bool dongu_var(oge_gost bas) {
2     oge_gost yavas = bas, hizli = bas;
3     while (hizli != NULL && hizli->bag != NULL) {
4         yavas = yavas->bag;          // 1 adım
5         hizli = hizli->bag->bag;     // 2 adım
▶         if (yavas == hizli) return true;
7     }
8     return false;
9 }
```

DEĞİŞKENLER

- `yavas = &düğüm(2)`
- `hizli = &düğüm(3)`

Kod İzleme: `dongu_var()` — tavşan ve kaplumbağa



5 → 3: döngü

`hizli` ve sonraki NULL değil: koşmaya devam.

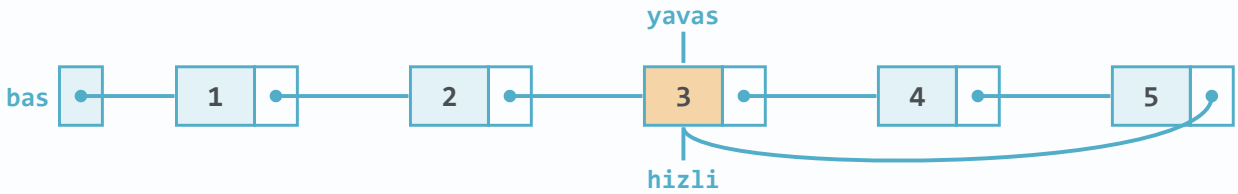
Adım 6 / 13

```
1 bool dongu_var(oge_gost bas) {
2     oge_gost yavas = bas, hizli = bas;
▶     while (hizli != NULL && hizli->bag != NULL) {
4         yavas = yavas->bag;          // 1 adım
5         hizli = hizli->bag->bag;     // 2 adım
6         if (yavas == hizli) return true;
7     }
8     return false;
9 }
```

DEĞİŞKENLER

- `yavas = &düğüm(2)`
- `hizli = &düğüm(3)`

Kod İzleme: `dongu_var()` — tavşan ve kaplumbağa



5 → 3: döngü

2. tur: kaplumbağa 1 adım → düğüm(3).

Adım 7 / 13

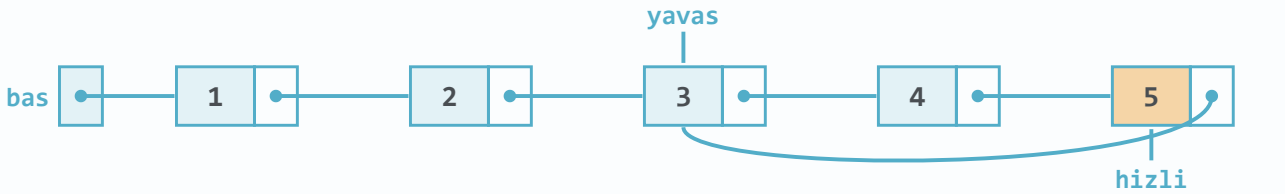
```
1 bool dongu_var(oge_gost bas) {
2     oge_gost yavas = bas, hizli = bas;
3     while (hizli != NULL && hizli->bag != NULL) {
▶         yavas = yavas->bag;          // 1 adım
5         hizli = hizli->bag->bag;     // 2 adım
6         if (yavas == hizli) return true;
7     }
8     return false;
9 }
```

DEĞİŞKENLER

`yavas = &düğüm(3)`

`hizli = &düğüm(3)`

Kod İzleme: `dongu_var()` — tavşan ve kaplumbağa



5 → 3: döngü

Tavşan 2 adım → düğüm(5).

Adım 8 / 13

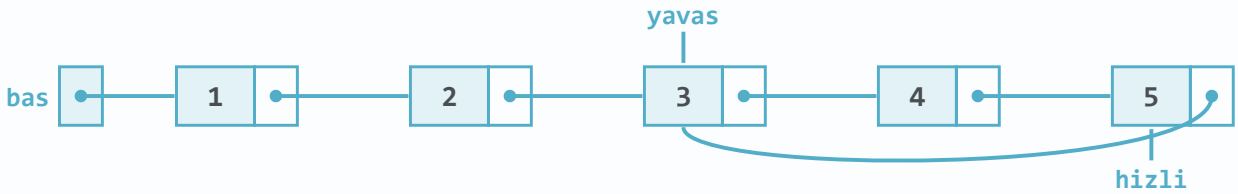
```
1 bool dongu_var(oge_gost bas) {
2     oge_gost yavas = bas, hizli = bas;
3     while (hizli != NULL && hizli->bag != NULL) {
4         yavas = yavas->bag;          // 1 adım
▶         hizli = hizli->bag->bag;     // 2 adım
6         if (yavas == hizli) return true;
7     }
8     return false;
9 }
```

DEĞİŞKENLER

`yavas = &düğüm(3)`

`hizli = &düğüm(5)`

Kod İzleme: `dongu_var()` — tavşan ve kaplumbağa



5 → 3: döngü

Aynı düğümde değiller; devam.

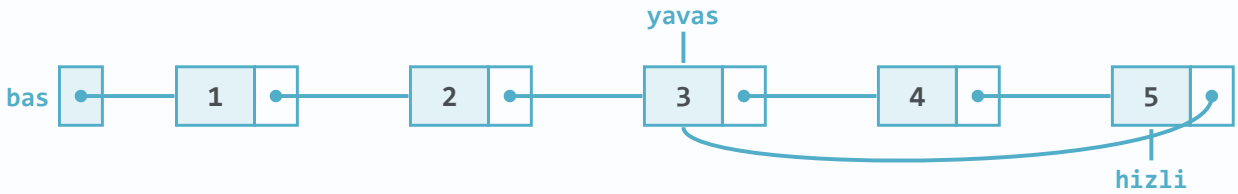
Adım 9 / 13

```
1 bool dongu_var(oge_gost bas) {
2     oge_gost yavas = bas, hizli = bas;
3     while (hizli != NULL && hizli->bag != NULL) {
4         yavas = yavas->bag;          // 1 adım
5         hizli = hizli->bag->bag;     // 2 adım
▶         if (yavas == hizli) return true;
7     }
8     return false;
9 }
```

DEĞİŞKENLER

- `yavas = &düğüm(3)`
- `hizli = &düğüm(5)`

Kod İzleme: `dongu_var()` — tavşan ve kaplumbağa



5 → 3: döngü

`hizli` ve sonraki NULL değil: koşmaya devam.

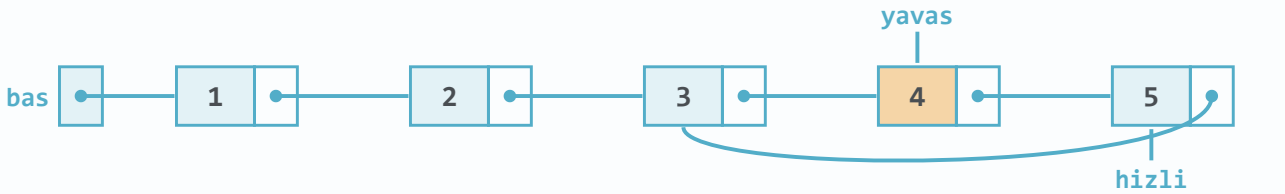
Adım 10 / 13

```
1 bool dongu_var(oge_gost bas) {
2     oge_gost yavas = bas, hizli = bas;
▶     while (hizli != NULL && hizli->bag != NULL) {
4         yavas = yavas->bag;          // 1 adım
5         hizli = hizli->bag->bag;     // 2 adım
6         if (yavas == hizli) return true;
7     }
8     return false;
9 }
```

DEĞİŞKENLER

- `yavas = &düğüm(3)`
- `hizli = &düğüm(5)`

Kod İzleme: `dongu_var()` — tavşan ve kaplumbağa



5 → 3: döngü

3. tur: kaplumbağa 1 adım → düğüm(4).

Adım 11 / 13

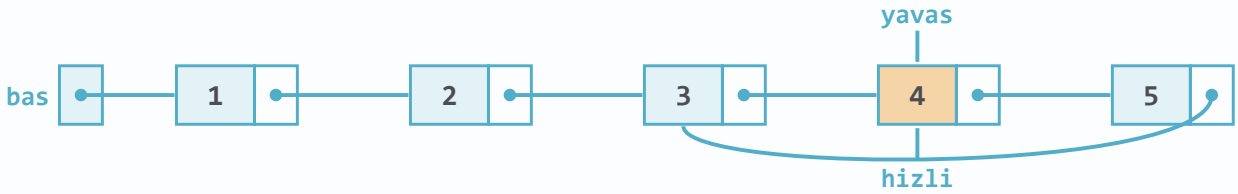
```
1 bool dongu_var(oge_gost bas) {
2     oge_gost yavas = bas, hizli = bas;
3     while (hizli != NULL && hizli->bag != NULL) {
▶         yavas = yavas->bag;          // 1 adım
5         hizli = hizli->bag->bag;     // 2 adım
6         if (yavas == hizli) return true;
7     }
8     return false;
9 }
```

DEĞİŞKENLER

`yavas = &düğüm(4)`

`hizli = &düğüm(5)`

Kod İzleme: `dongu_var()` — tavşan ve kaplumbağa



5 → 3: döngü

Tavşan 2 adım → düğüm(4).

Adım 12 / 13

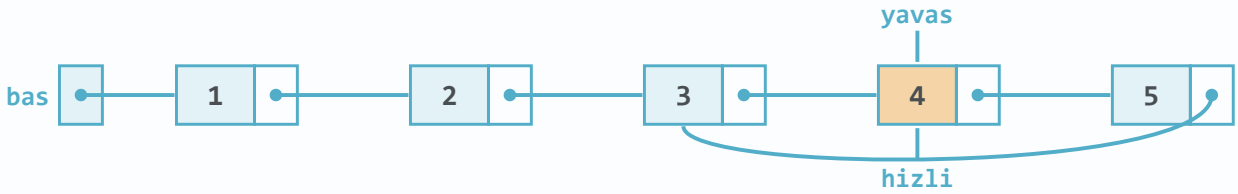
```
1 bool dongu_var(oge_gost bas) {
2     oge_gost yavas = bas, hizli = bas;
3     while (hizli != NULL && hizli->bag != NULL) {
4         yavas = yavas->bag;          // 1 adım
▶         hizli = hizli->bag->bag;     // 2 adım
6         if (yavas == hizli) return true;
7     }
8     return false;
9 }
```

DEĞİŞKENLER

yavas = &düğüm(4)

hizli = &düğüm(4)

Kod İzleme: `dongu_var()` — tavşan ve kaplumbağa



5 → 3: döngü

Buluştular! Tavşan döngüde tur atıp kaplumbağaya yetişti → döngü var, `true` .

Adım 13 / 13

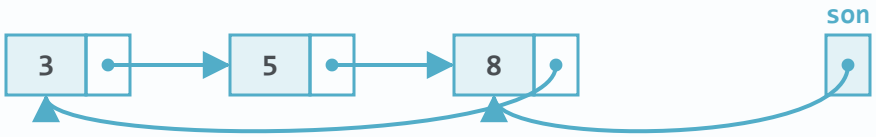
```
1 bool dongu_var(oge_gost bas) {
2     oge_gost yavas = bas, hizli = bas;
3     while (hizli != NULL && hizli->bag != NULL) {
4         yavas = yavas->bag;          // 1 adım
5         hizli = hizli->bag->bag;     // 2 adım
▶         if (yavas == hizli) return true;
7     }
8     return false;
9 }
```

DEĞİŞKENLER

`yavas = &düğüm(4)`

`hizli = &düğüm(4)`

Dairesel (circular) liste: son düğüm başı gösterir




```c
int d_uzunluk(oge_gost son) {
    int n = 0;
    oge_gost p = son;
    if (son == NULL) return 0;    // boş liste
    do {
        n++;
        p = p->bag;
    } while (p != son);           // tur tamamlandı
    return n;
}
```

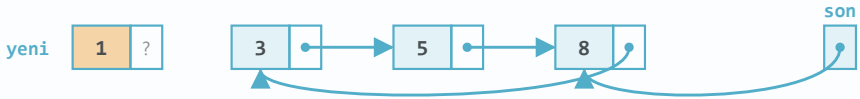
Neden `son`?

`son→bag` ilk düğümdür: hem başa hem sona ekleme O(1).

NULL yok

Gezinmenin bitişi `p == NULL` değil, başa dönmek; bu yüzden `do-while`.

Dairesel listede başa ve sona ekleme



3 → 5 → 8 → (3), `son` = 8. Yeni düğüm 1 eklenecek.

Adım 1 / 4

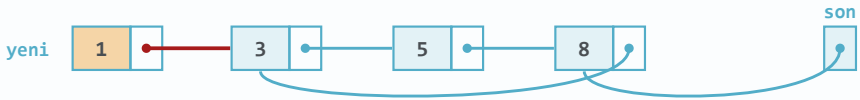
```
▶ void d_basa_ekle(oge_gost *son, oge_gost yeni) {
2     if (*son == NULL) {            // boş liste
3         *son = yeni;
4         yeni->bag = yeni;         // kendini gösterir
5     } else {
6         yeni->bag = (*son)->bag;  // eski ilk düğüm
7         (*son)->bag = yeni;
8     }
9 }
```

DEĞİŞKENLER

`*son = &düğüm(8)` `yeni = &düğüm(1)`

Sona ekleme: `d_basa_ekle(son, yeni); *son = yeni;` — başa ve sona ekleme aynı bağ işlemidir, fark yalnızca son işaretçisindedir.

Dairesel listede başa ve sona ekleme



`yeni→bag = son→bag` : yeni düğüm eski ilk düğümü (3) gösterir.

Adım 2 / 4

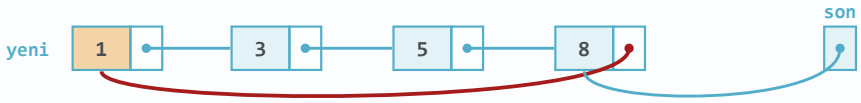
```
1 void d_basa_ekle(oge_gost *son, oge_gost yeni) {
2     if (*son == NULL) {            // boş liste
3         *son = yeni;
4         yeni->bag = yeni;         // kendini gösterir
5     } else {
▶         yeni->bag = (*son)->bag;  // eski ilk düğüm
7         (*son)->bag = yeni;
8     }
9 }
```

DEĞİŞKENLER

`*son = &düğüm(8)` `yeni = &düğüm(1)`

Sona ekleme: `d_basa_ekle(son, yeni); *son = yeni;` — başa ve sona ekleme aynı bağ işlemidir, fark yalnızca son işaretçisindedir.

Dairesel listede başa ve sona ekleme



`son→bag = yeni` : 8 artık 1'i gösterir. **Başa ekleme bitti**: 1 → 3 → 5 → 8.

Adım 3 / 4

```
1 void d_basa_ekle(oge_gost *son, oge_gost yeni) {
2     if (*son == NULL) {            // boş liste
3         *son = yeni;
4         yeni->bag = yeni;         // kendini gösterir
5     } else {
6         yeni->bag = (*son)->bag;  // eski ilk düğüm
▶         (*son)->bag = yeni;
8     }
9 }
```

DEĞİŞKENLER

`*son = &düğüm(8)` `yeni = &düğüm(1)`

Sona ekleme: `d_basa_ekle(son, yeni); *son = yeni;` — başa ve sona ekleme aynı bağ işlemidir, fark yalnızca son işaretçisindedir.

Dairesel listede başa ve sona ekleme



Sona ekleme = başa ekle + `*son = yeni` : son artık 1; sıra 3 → 5 → 8 → 1. İkisi de **O(1)**.

Adım 4 / 4

```
1 void d_basa_ekle(oge_gost *son, oge_gost yeni) {
2     if (*son == NULL) {           // boş liste
3         *son = yeni;
4         yeni->bag = yeni;        // kendini gösterir
5     } else {
6         yeni->bag = (*son)->bag;  // eski ilk düğüm
7         (*son)->bag = yeni;
8     }
▶ }
```

DEĞİŞKENLER

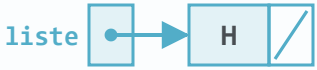
`*son = &düğüm(1)` `yeni = &düğüm(1)`

Sona ekleme: `d_basa_ekle(son, yeni); *son = yeni;` — başa ve sona ekleme aynı bağ işlemidir, fark yalnızca son işaretçisindedir.

Başlık (header / sentinel) düğümü



dolu liste: H → 3 → 8



boş liste: yalnız H

```
// başlıklı listede sıralı ekleme: özel durum yok
void b_sirali_ekle(oge_gost baslik, oge_gost yeni) {
    oge_gost onceki = baslik;       // asla NULL değil
    while (onceki->bag != NULL &&
           onceki->bag->deger < yeni->deger)
        onceki = onceki->bag;
    yeni->bag = onceki->bag;
    onceki->bag = yeni;
}
```

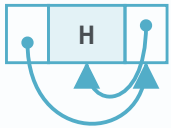
Fikir

Listenin başında veri taşımayan, hiç silinmeyen bir düğüm. Boş liste = yalnızca başlık.

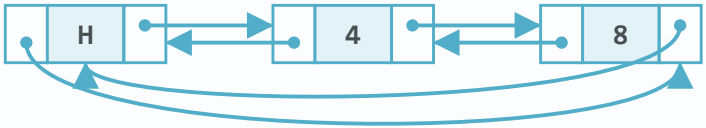
Kazanç

Her gerçek düğümün bir öncesi vardır: `*bas` ve `if (onceki == NULL)` özel durumu kalkar.

Çift yönlü bağlı (doubly linked) liste



boş: H kendini gösterir



H ⇄ 4 ⇄ 8 ⇄ H

```c
typedef struct doge *doge_gost;
struct doge {
    doge_gost sol;   // önceki (llink)
    int deger;
    doge_gost sag;   // sonraki (rlink)
};
// değişmez: p == p->sol->sag == p->sag->sol
```

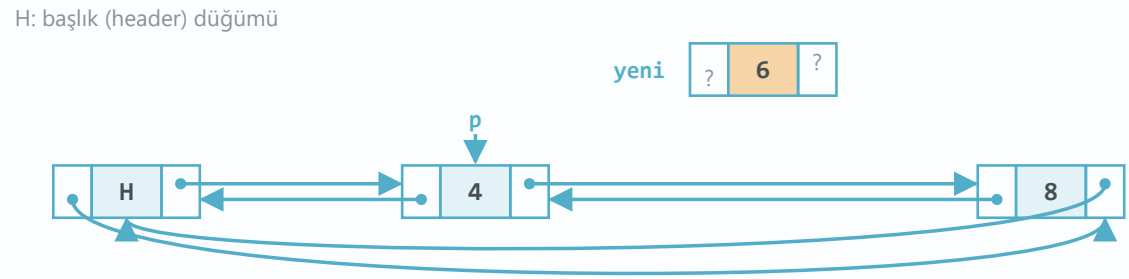
Artı

- + İki yönde gezinme
- + Bilinen düğümü O(1) silme
- + Öncesine O(1) ekleme

Eksi

- – Düğüm başına 2 işaretçi
- – Her işlemde 4 bağ
- – Bağ hatası daha kolay

Kod İzleme: `cift_ekle(p, yeni)`



Başlık (header) düğümlü dairesel çift yönlü bağlı liste: H ⇄ 4 ⇄ 8 ⇄ H. 6, 4'ün sağına eklenecek.

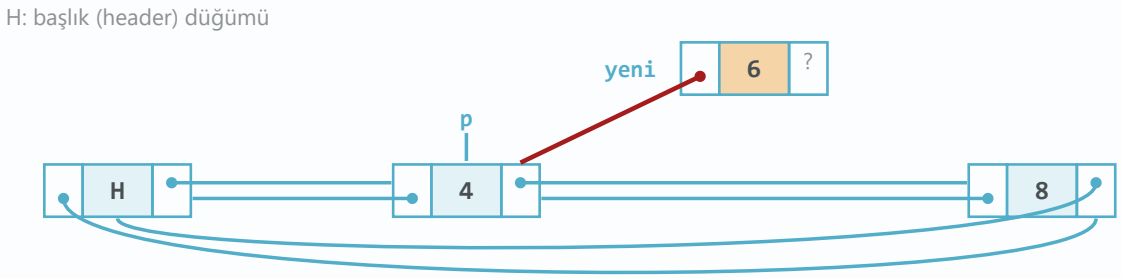
Adım 1 / 6

```
1 // p düğümünün sağına yeni düğümü ekler
▶ void cift_ekle(doge_gost p, doge_gost yeni) {
3     yeni->sol = p;
4     yeni->sag = p->sag;
5     p->sag->sol = yeni;
6     p->sag = yeni;
7 }
```

DEĞİŞKENLER

- `p = &düğüm(4)`
- `yeni = &düğüm(6)`
- `p→sag = &düğüm(8)`
- `yeni→sag = ?`

Kod İzleme: `cift_ekle(p, yeni)`



`yeni→sol = p` : yeni düğümün sol bağı 4'ü gösterir.

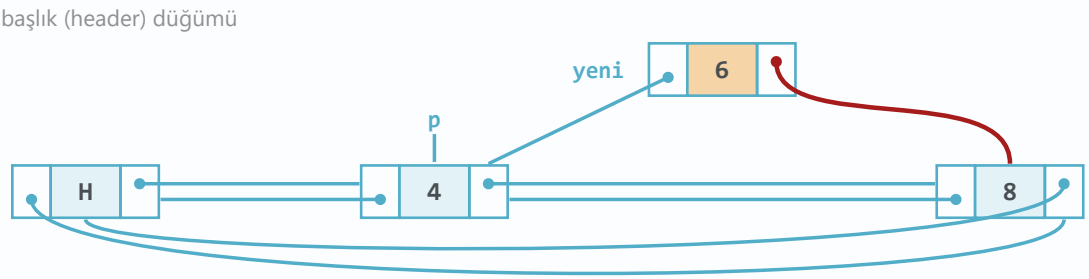
Adım 2 / 6

```
1 // p düğümünün sağına yeni düğümü ekler
2 void cift_ekle(doge_gost p, doge_gost yeni) {
▶     yeni->sol = p;
4     yeni->sag = p->sag;
5     p->sag->sol = yeni;
6     p->sag = yeni;
7 }
```

DEĞİŞKENLER

- `p = &düğüm(4)`
- `yeni = &düğüm(6)`
- `p→sag = &düğüm(8)`
- `yeni→sag = ?`

Kod İzleme: `cift_ekle(p, yeni)`



`yeni→sag = p→sag` : sağ bağı 8'i gösterir. Liste henüz değişmedi, yalnız yeni düğüm hazırlandı.

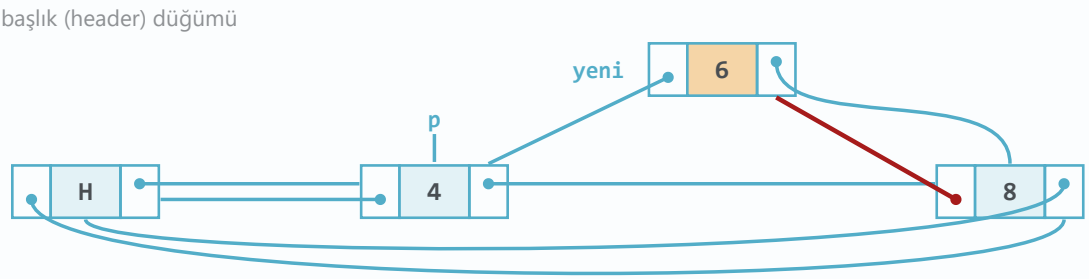
Adım 3 / 6

```
1 // p düğümünün sağına yeni düğümü ekler
2 void cift_ekle(doge_gost p, doge_gost yeni) {
3     yeni->sol = p;
▶    yeni->sag = p->sag;
5     p->sag->sol = yeni;
6     p->sag = yeni;
7 }
```

DEĞİŞKENLER

- `p = &düğüm(4)`
- `yeni = &düğüm(6)`
- `p→sag = &düğüm(8)`
- `yeni→sag = &düğüm(8)`

Kod İzleme: `cift_ekle(p, yeni)`



`p→sag→sol = yeni` : 8'in sol bağı artık 6. (p->sag hâlâ 8 olduğu için bu satır, 4. atamadan (`p→sag = yeni`) **sonra** gelemezdi.)

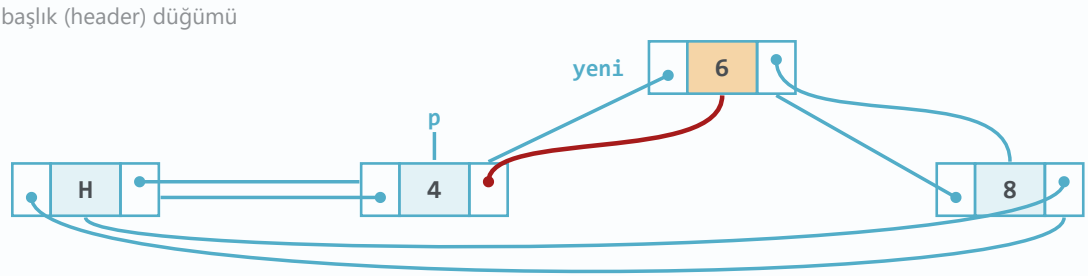
Adım 4 / 6

```
1 // p düğümünün sağına yeni düğümü ekler
2 void cift_ekle(doge_gost p, doge_gost yeni) {
3     yeni->sol = p;
4     yeni->sag = p->sag;
▶     p->sag->sol = yeni;
6     p->sag = yeni;
7 }
```

DEĞİŞKENLER

- `p = &düğüm(4)`
- `yeni = &düğüm(6)`
- `p→sag = &düğüm(8)`
- `yeni→sag = &düğüm(8)`

Kod İzleme: `cift_ekle(p, yeni)`



`p→sag = yeni` : son bağ. Dört bağ güncellendi; özel durum yok: **O(1)**.

Adım 5 / 6

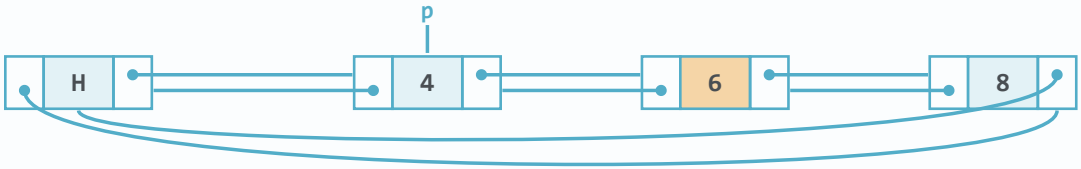
```
1 // p düğümünün sağına yeni düğümü ekler
2 void cift_ekle(doge_gost p, doge_gost yeni) {
3     yeni->sol = p;
4     yeni->sag = p->sag;
5     p->sag->sol = yeni;
▶     p->sag = yeni;
7 }
```

DEĞİŞKENLER

- `p = &düğüm(4)`
- `yeni = &düğüm(6)`
- `p→sag = &düğüm(6)`
- `yeni→sag = &düğüm(8)`

Kod İzleme: `cift_ekle(p, yeni)`

H: başlık (header) düğümü



Sonuç: H ⇄ 4 ⇄ 6 ⇄ 8 ⇄ H. Başlık düğümü sayesinde boş listede de, sonda da aynı kod çalışır.

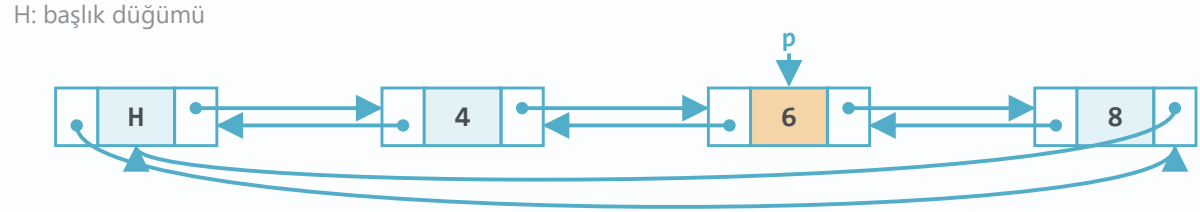
Adım 6 / 6

```
1 // p düğümünün sağına yeni düğümü ekler
2 void cift_ekle(doge_gost p, doge_gost yeni) {
3     yeni->sol = p;
4     yeni->sag = p->sag;
5     p->sag->sol = yeni;
▶     p->sag = yeni;
7 }
```

DEĞİŞKENLER

- `p = &düğüm(4)`
- `yeni = &düğüm(6)`
- `p→sag = &düğüm(6)`
- `yeni→sag = &düğüm(8)`

Çift yönlü bağlı listede silme: öncesi aranmaz



H ⇄ 4 ⇄ 6 ⇄ 8 ⇄ H; `p` = düğüm(6). Öncesi `p→sol` ile zaten elde.

Adım 1 / 4

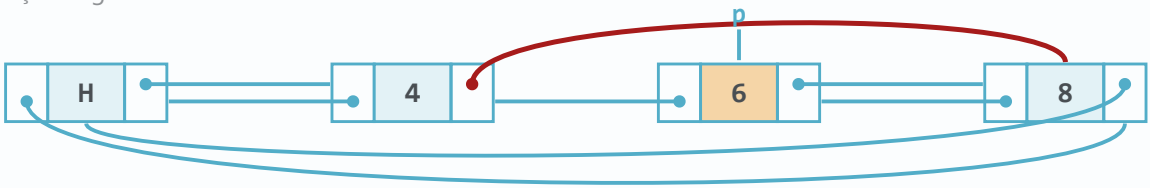
```
1 void cift_sil(doge_gost baslik, doge_gost p) {
▶     if (p == baslik) {       // başlık silinmez
3         fprintf(stderr, "Baslik silinemez\n");
4         return;
5     }
6     p->sol->sag = p->sag;
7     p->sag->sol = p->sol;
8     free(p);
9 }
```

DEĞİŞKENLER

`p = &düğüm(6)`

Çift yönlü bağlı listede silme: öncesi aranmaz

H: başlık düğümü



`p->sol->sag = p->sag` : 4'ün sağı artık 8.

Adım 2 / 4

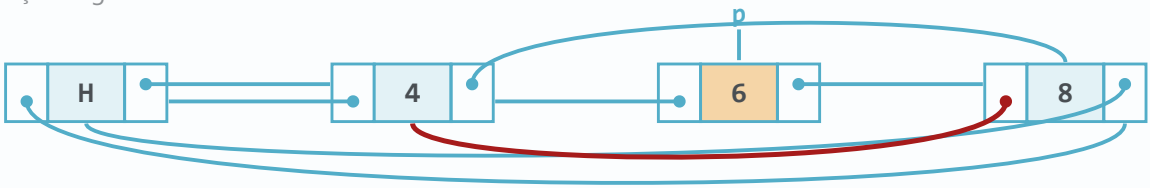
```
1 void cift_sil(doge_gost baslik, doge_gost p) {
2     if (p == baslik) {       // başlık silinmez
3         fprintf(stderr, "Baslik silinemez\n");
4         return;
5     }
▶     p->sol->sag = p->sag;
7     p->sag->sol = p->sol;
8     free(p);
9 }
```

DEĞİŞKENLER

`p = &düğüm(6)`

Çift yönlü bağlı listede silme: öncesi aranmaz

H: başlık düğümü



`p→sag→sol = p→sol` : 8'in solu artık 4.

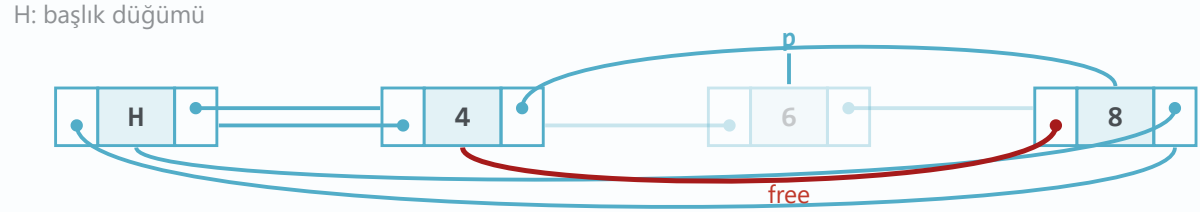
Adım 3 / 4

```
1 void cift_sil(doge_gost baslik, doge_gost p) {
2     if (p == baslik) {       // başlık silinmez
3         fprintf(stderr, "Baslik silinemez\n");
4         return;
5     }
6     p->sol->sag = p->sag;
▶     p->sag->sol = p->sol;
8     free(p);
9 }
```

DEĞİŞKENLER

`p = &düğüm(6)`

Çift yönlü bağlı listede silme: öncesi aranmaz



`free(p)`. Arama yok: **O(1)**. Başlık sayesinde ilk/son düğüm için ayrı kod gerekmez.

Adım 4 / 4

```
1 void cift_sil(doge_gost baslik, doge_gost p) {
2     if (p == baslik) {       // başlık silinmez
3         fprintf(stderr, "Baslik silinemez\n");
4         return;
5     }
6     p->sol->sag = p->sag;
7     p->sag->sol = p->sol;
▶     free(p);
9 }
```

DEĞİŞKENLER

`p = (serbest)`

BELLEK YÖNETİMİ

Kullanılabilir düğüm listesi (available space list)

```
oge_gost avail = NULL;          // kullanılabilir düğümler

oge_gost get_node(void) {
    oge_gost p;
    if (avail != NULL) {          // önce listeden al
        p = avail;
        avail = avail->bag;
    } else {                      // yoksa sistemden iste
        p = malloc(sizeof *p);
        if (p == NULL) exit(1);
    }
    return p;
}

void ret_node(oge_gost p) {       // free yerine
    p->bag = avail;
    avail = p;
}
```

Fikir (Horowitz)

Silinen düğümler `free` yerine `avail` zincirine eklenir; yeni düğüm önce oradan alınır.

Neden?

Sık ekle-sil yapan programlarda `malloc` / `free` çağrılarını azaltır; düğüm alma/iade O(1).

Sınırlama

avail'deki düğümler program bitene dek heap belleğe dönmez; düğüm türü tekse işe yarar.

`get_node` ve `ret_node` adım adım



`avail` zincirinde iki boş düğüm var; sağda kullanımda olan düğüm(9).

Adım 1 / 5

Aslında `avail` bir **yığın**dır: `get_node` = al (pop), `ret_node` = ekle (push).

`get_node` ve `ret_node` adım adım



`get_node()` : avail boş değil → `p = avail` . malloc çağrılmadı.

Adım 2 / 5

Aslında `avail` bir **yığın**dır: `get_node` = al (pop), `ret_node` = ekle (push).

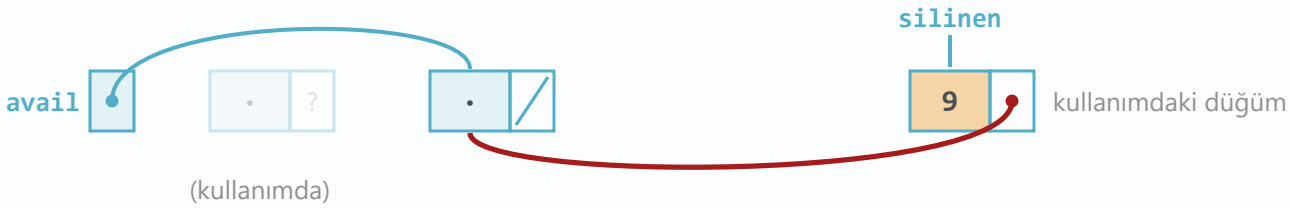
`get_node` ve `ret_node` adım adım



`avail = avail→bag` : zincirin başı ilerledi; p artık çağırana ait.

Aslında `avail` bir **yığın**dır: `get_node` = al (pop), `ret_node` = ekle (push).

`get_node` ve `ret_node` adım adım

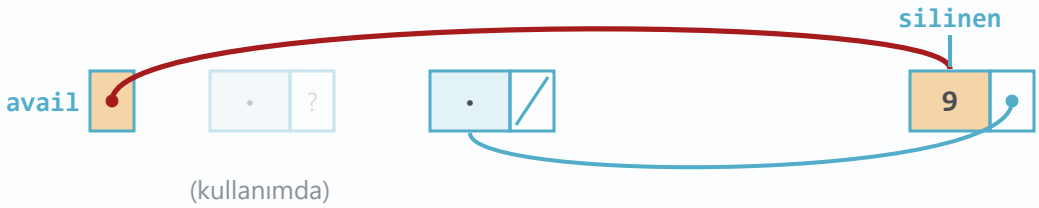


`ret_node(silinen)` : `silinen→bag = avail` .

Adım 4 / 5

Aslında `avail` bir **yığın**dır: `get_node` = al (pop), `ret_node` = ekle (push).

`get_node` ve `ret_node` adım adım

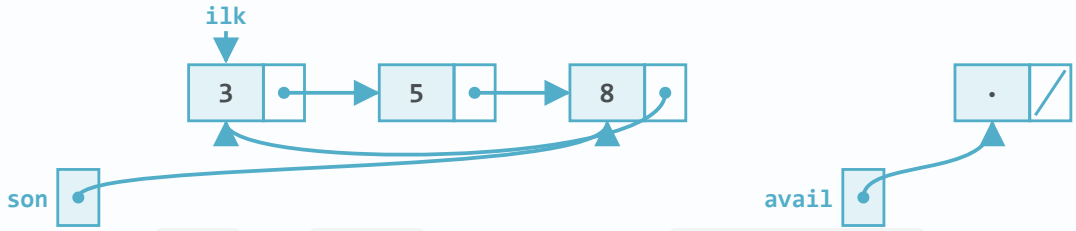


`avail = silinen` : düğüm yeniden kullanıma hazır. İkisi de bağlı yığının ekle/al işlemi gibi **O(1)**.

Adım 5 / 5

Aslında `avail` bir **yığın**dır: `get_node` = al (pop), `ret_node` = ekle (push).

Dairesel listeyi tek hamlede iade (Horowitz: `cerase`)



3 → 5 → 8 → (3), `son` = 8; `avail` 'de bir düğüm var. `ilk = son→bag` = 3.

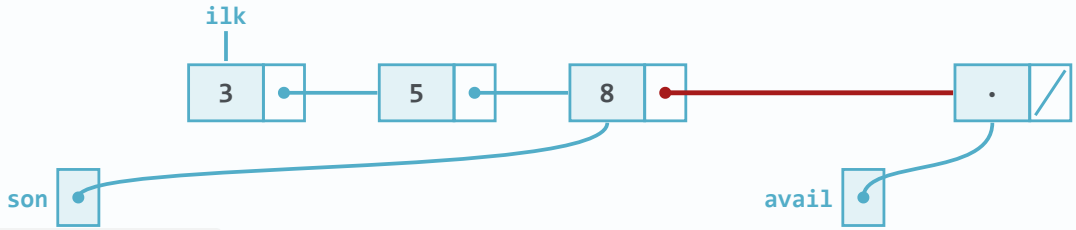
Adım 1 / 4

```
1  // dairesel listeyi O(1)'de avail'e iade et
2  void d_iade(oge_gost *son) {
3      oge_gost ilk;
4      if (*son != NULL) {
▶          ilk = (*son)->bag;      // ilk düğüm
6          (*son)->bag = avail;    // halkayı aç
7          avail = ilk;
8          *son = NULL;
9      }
10 }
```

DEĞİŞKENLER

`ilk` = **&düğüm(3)** `*son` = **&düğüm(8)** `avail` = **&düğüm(·)**

Dairesel listeyi tek hamlede iade (Horowitz: `cerase`)



`son→bag = avail` : halka açıldı, sonu avail zincirine eklendi.

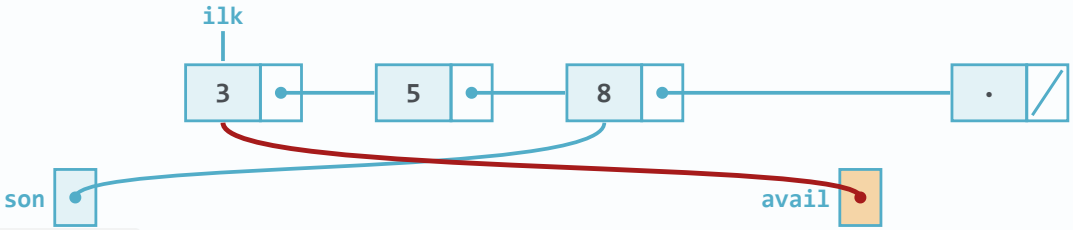
Adım 2 / 4

```
1  // dairesel listeyi O(1)'de avail'e iade et
2  void d_iade(oge_gost *son) {
3      oge_gost ilk;
4      if (*son != NULL) {
5          ilk = (*son)->bag;      // ilk düğüm
▶          (*son)->bag = avail;    // halkayı aç
7          avail = ilk;
8          *son = NULL;
9      }
10 }
```

DEĞİŞKENLER

`ilk` = **&düğüm(3)** `*son` = **&düğüm(8)** `avail` = **&düğüm(·)**

Dairesel listeyi tek hamlede iade (Horowitz: `cerase`)



`avail = ilk` : tüm liste avail'in önüne geçti.

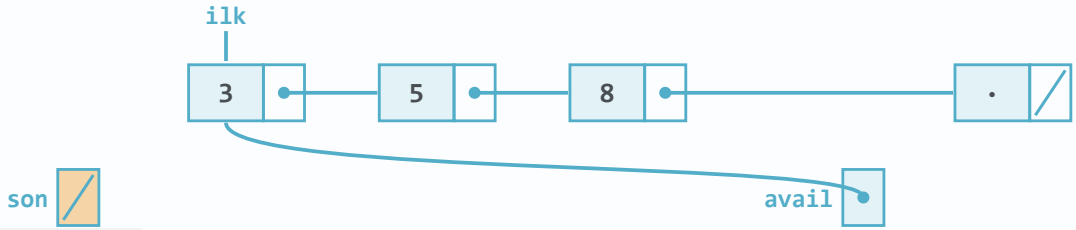
Adım 3 / 4

```
1  // dairesel listeyi O(1)'de avail'e iade et
2  void d_iade(oge_gost *son) {
3      oge_gost ilk;
4      if (*son != NULL) {
5          ilk = (*son)->bag;      // ilk düğüm
6          (*son)->bag = avail;    // halkayı aç
▶          avail = ilk;
8          *son = NULL;
9      }
10 }
```

DEĞİŞKENLER

`ilk` = **&düğüm(3)** `*son` = **&düğüm(8)** `avail` = **&düğüm(3)**

Dairesel listeyi tek hamlede iade (Horowitz: `cerase`)



`*son = NULL`. Uzunluk ne olursa olsun **O(1)**; tek yönlü bağlı düz listede sonu bulmak O(n) sürerdi.

Adım 4 / 4

```
1  // dairesel listeyi O(1)'de avail'e iade et
2  void d_iade(oge_gost *son) {
3      oge_gost ilk;
4      if (*son != NULL) {
5          ilk = (*son)->bag;      // ilk düğüm
6          (*son)->bag = avail;    // halkayı aç
7          avail = ilk;
▶          *son = NULL;
9      }
10 }
```

DEĞİŞKENLER

`ilk = &düğüm(3)` `*son = NULL` `avail = &düğüm(3)`

LABORATUVAR · C KODU ADIM ADIM

Laboratuvar: Tek Yönlü Bağlı Liste

Sıralı ekleme ve silme

Girdi (sayı = sıralı ekle, sil 30) `30, 10, 20, 25, sil 10, sil 5` Uygula

KOD: sirali_ekle

```
 1 void sirali_ekle(oge_gost *bas,
 2                  oge_gost yeni) {
 3   oge_gost onceki = NULL, p = *bas;
 4   while (p != NULL &&
 5          p->deger < yeni->deger) {
 6     onceki = p;
 7     p = p->bag;
 8   }
 9   yeni->bag = p;
10   if (onceki == NULL) *bas = yeni;  // başa
11   else onceki->bag = yeni;   // araya/sona
12 }
```

boş liste (NULL)

`bas` NULL

ÇIKTI (henüz yok)

Başlangıçta liste boş: `bas = NULL`. Sayılar `sirali_ekle` ile artan sırayı bozmadan eklenir; `sil` değeri arar ve düğümü listeden çıkarır.

KAPANIŞ

Karmaşıklık özeti

İşlem	Dizi	Tek yönlü bağlı (baş)	Dairesel (son)	Çift yönlü bağlı (başlıklı)
i. öğeye erişim	O(1)	O(i)	O(i)	O(i)
Başa ekleme	O(n)	O(1)	O(1)	O(1)
Sona ekleme	O(1)*	O(n)	O(1)	O(1)
Bilinen düğümden sonra ekleme	O(n)	O(1)	O(1)	O(1)
Bilinen düğümü silme	O(n)	O(n)†	O(n)†	O(1)
Değer arama	O(n)	O(n)	O(n)	O(n)
Birleştirme	O(n+m)	O(n)	O(1)	O(1)
Düğüm başına ek bellek	—	1 işaretçi	1 işaretçi	2 işaretçi

* kapasite doluysa yeniden boyutlandırma gerekir · † öncesini bulmak için gezinme; öncesi biliniyorsa O(1). Floyd, orta düğüm, ters çevirme: O(n) zaman, O(1) ek bellek.

KAPANIŞ

Sık yapılan hatalar

1. Bağı korumadan üzerine yazmak: `p→bag = yeni` önce yazılırsa listenin kalanı kaybolur.
2. Baş değişen yordama işaretçiyi **değerle** vermek (`oge_gost bas`); main'deki liste güncellenmez.
3. `malloc` sonucunu denetlememek; `free`'yi unutmak (sızıntı) ya da iki kez çağırmak.
4. free edilmiş düğümün `bag` alanını okumak; önce sonraki saklanmalı, sonra bırakılmalı.
5. Boş liste, tek düğüm, baş/son düğüm durumlarını test etmemek.
6. Dairesel listede `p ≠ NULL` ile gezinmek: döngü hiç bitmez.

KAPANIŞ

Alıştırmalar

1. Sıralı iki listeyi, düğüm ayırmadan tek sıralı listede birleştirin.
2. Listenin sondan k. düğümünü tek geçişte bulun.
3. Sıralı listedeki tekrarları silin; silinenleri `ret_node` ile iade edin.
4. `liste_devrik_al`'ı özyinelemeli yazın. Ek bellek ne olur?
5. Floyd ile döngü bulunduktan sonra döngünün uzunluğunu hesaplayın.

▶ **Cevaplar / ipuçları**

Hocadan "Ödev kâğıdı benim kâğıdımla üst üste getirilip ışığa tutulduğunda… (fark görülmemeli)"

KAPANIŞ

Bölüm özeti

Düğüm + bağ

Liste öğeleri dağınık durur; sıra `bag` alanlarıyla kurulur. Boş liste = NULL.

Bağ disiplini

Önce korunur, sonra bağlanır, en son bırakılır (`free`). Baş değişecekse adresle aktarılır.

Maliyetler

Konum biliniyorsa ekle/sil O(1); erişim ve arama O(n).

Klasikler

Ters çevirme, birleştirme, orta düğüm, Floyd: tek geçiş, O(1) ek bellek.

Türler

Dairesel (son işaretçisi), çift yönlü bağlı (iki yön), başlık düğümü (özel durum yok).

Bellek

avail listesi düğümleri yeniden kullanır; dairesel liste O(1)'de iade edilir.

KAPANIŞ

Kaynaklar ve devam

- E. Horowitz, S. Sahni, S. Anderson-Freed, *Fundamentals of Data Structures in C*, 2. bs., Bölüm 4 (listeler, avail, dairesel ve çift yönlü bağlı).
- T. H. Cormen vd., *Introduction to Algorithms*, Bölüm 10.2 (sentinel'li listeler).
- D. E. Knuth, *The Art of Computer Programming* Cilt 2, Alıştırma 3.1-6 (Floyd döngü tespiti).

← Bölüm 3: Özyineleme, İkili Arama ve Geri İzleme

Ders sayfası

Bölüm 5: Polinomlar ve Seyrek Matrisler

Son

Bölüm 4: Bağlı Listeler

Sonraki bölüm: Bölüm 5: Polinomlar ve Seyrek Matrisler · Ders sayfası